HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

SCHOOL OF ELECTRICAL & ELECTRONIC ENGINEERING

PROJECT REPORT

Monitoring Audio Compression in Real Time with Interactive Dashboards

Class ID: 168224

Project ID: 25219

Team members: Bui Quynh Hoa 202414627

Hoang Thi Phuong Nga 202414648

Instructor: Dr. rer. nat. Pham Van Tien

Hanoi, 04/2026

TABLE OF CONTENTS

TABLE OF CONTENTS 2

I. PROBLEM STATEMENT AND MOTIVATION 4

II. BACKGROUND 5

- **1. Audio Codecs Overview** 5
- **2. Evaluation Metrics** 5

III. SYSTEM DESIGN 7

- **1. Architecture Overview** 7
- **2. Core Components** 7

IV. METHODOLOGY 8

- **1. Processing Pipeline** 8
- **2. Mathematical Metrics** 8

V. IMPLEMENTATION DETAILS 10

- **1. Technology Stack** 10
- **2. Project Structure** 10
- **3. Dataset** 10

EVALUATION 12

- **1. Evaluation Metrics Summary** 12
- **2. Experimental Setup** 12

VI. RESULTS & ANALYSIS 14

- **1. Sample.wav** 14
  - **1.1. Quantitative Analysis** 14
    - **1.1.1. Signal-to-Noise Ratio (SNR) and Audio Fidelity** 14
    - **1.1.2. Compression Efficiency** 16
    - **1.1.3. System Latency (Real-time Performance)** 17
  - **1.2. Qualitative Analysis** 19
    - **1.2.1. Spectrogram Analysis** 19
    - **1.2.2. Waveform Comparison** 20
- **2. Speech_test.wav** 21
  - **2.1. Quantitative Analysis** 21
    - **2.1.1. Signal-to-Noise Ratio (SNR) and Audio Fidelity** 21
    - **2.1.2. Compression Efficiency** 22
    - **2.1.3. System Latency (Real-time Performance)** 24
  - **2.2. Qualitative Analysis** 25

2.2.1. Spectrogram Analysis 25
2.2.2. Waveform Comparison 27
3. Music_test.wav 27
3.1. Quantitative Analysis 27
3.1.1. Signal-to-Noise Ratio (SNR) and Audio Fidelity 27
3.1.2. Compression Efficiency 29
3.1.3. System Latency (Real-time Performance) 31
3.2. Qualitative Analysis 32
3.2.1. Spectrogram Analysis 32
3.2.2. Waveform Comparison 33
4. Percussion_test.wav 34
4.1. Quantitative Analysis 34
4.1.1. Signal-to-Noise Ratio (SNR) and Audio Fidelity 34
4.1.2. Compression Efficiency 36
4.1.3. System Latency (Real-time Performance) 38
4.2. Qualitative Analysis 40
4.2.1. Spectrogram Analysis 40
4.2.2. Waveform Comparison 41
5. Noise_test.wav 41
5.1. Quantitative Analysis 41
5.1.1. Signal-to-Noise Ratio (SNR) and Audio Fidelity 42
5.1.2. Compression Efficiency 44
5.1.3. System Latency (Real-time Performance) 45
5.2. Qualitative Analysis 47
5.2.1. Spectrogram Analysis 47
5.2.2. Waveform Comparison 49
VII. LIMITATIONS 50
VIII. FUTURE WORK 51
IX. CONCLUSION 52
X. REPRODUCIBILITY 53
XI. TEAM CONTRIBUTION STATEMENT 54
XII. TABLE OF FIGURES AND TABLES 55
XIII. SUPPLEMENTARY MATERIALS 58

I. PROBLEM STATEMENT AND MOTIVATION

In modern communication systems such as VoIP, live streaming, and video conferencing, real-time audio transmission requires a strict balance between three critical factors: signal quality, bandwidth (bitrate), and latency.

Selecting the appropriate codec and bitrate is often challenging due to the lack of visual monitoring tools capable of simultaneously analyzing objective parameters (SNR, PSNR) and subjective parameters (spectral distortion via spectrograms) in a real-time environment. This project was developed to build a comprehensive monitoring dashboard, allowing users or system engineers to intuitively analyze, compare, and optimize the performance of popular audio compression standards (MP3, AAC, Opus, OGG) to make optimal network bandwidth decisions.

This project addresses this gap by designing and implementing a comprehensive **real-time audio compression monitoring system** with an interactive web-based dashboard. The system enables system engineers, researchers, and students to:

- Visually observe and compare the compression behavior of four industry-standard audio codecs MP3, AAC, Opus, and OGG Vorbis operating simultaneously on the same audio stream.
- Quantitatively evaluate codec performance through objective metrics (SNR, PSNR, compression ratio, effective bitrate, encode/decode latency) computed on each audio chunk as it is processed.
- Qualitatively assess frequency-domain distortion by visually comparing original and reconstructed spectrograms and mel-spectrograms in real time.
- Interactively sweep bitrate parameters (32 to 320 kbps) to identify optimal operating points for specific use-case requirements.

The system is specifically designed to highlight the engineering trade-offs that determine codec suitability: Opus excels in low-latency real-time communication scenarios, while AAC provides superior perceptual quality at moderate bitrates for music streaming. MP3, despite its age, maintains broad hardware compatibility. OGG Vorbis offers competitive open-source performance. The dashboard makes these distinctions immediately observable, providing an educational and practical tool for multimedia system design.

II. BACKGROUND

1. Audio Codecs Overview

This system evaluates four widely used codecs:

MP3 (MPEG-1 Audio Layer III)

MP3 was standardized in 1993 as part of the MPEG-1 specification and remains one of the most widely deployed audio formats. Its encoder applies a 32-band polyphase filterbank followed by a Modified Discrete Cosine Transform (MDCT) to decompose the signal into frequency subbands. Bit allocation across subbands is guided by a psychoacoustic model (ISO Model I or Model II) that computes simultaneous and temporal masking thresholds. Quantization noise is shaped to remain below the masking threshold in each subband. While highly compatible, MP3 suffers from pre-echo artifacts at low bitrates and does not match the efficiency of more modern codecs.

AAC (Advanced Audio Coding)

AAC, standardized in MPEG-2 and MPEG-4, supersedes MP3 with several architectural improvements: a pure MDCT filterbank, a more sophisticated psychoacoustic model, temporal noise shaping (TNS), and perceptual noise substitution (PNS). These features allow AAC to achieve equivalent perceptual quality to MP3 at approximately 70% of the bitrate. AAC is the default format for Apple platforms, YouTube, and most streaming services.

OGG Vorbis

OGG Vorbis is a fully open-source, patent-free codec developed by the Xiph.Org Foundation. It employs a channel-coupled MDCT with variable-length encoding and a floor-and-residue architecture for spectral representation. Vorbis uses a psychoacoustic model based on a masking curve derived from a modified version of the ISO model. At comparable bitrates, Vorbis generally matches or exceeds MP3 quality, and its fully open nature makes it attractive for open-source applications.

Opus

Opus is a modern, versatile codec standardized by the IETF (in 2012, designed specifically for real-time internet communication. Opus is unique in combining two underlying coding modes: SILK) for speech at low bitrates, and CELT (a low-latency transform codec based on MDCT) for music and general audio. The codec dynamically selects and blends between modes. Opus supports bitrates from 6 kbps to 510 kbps, frame sizes from 2.5 ms to 60 ms, and achieves state-of-the-art quality at low bitrates. Its algorithmic delay is approximately 26.5 ms, far below MP3's minimum of ~100 ms, making it the standard choice for WebRTC and VoIP applications.

2. Evaluation Metrics

To evaluate compression performance, the system uses both objective and perceptual metrics:

- SNR (Signal-to-Noise Ratio): Measures signal fidelity
- PSNR (Peak Signal-to-Noise Ratio): Measures peak distortion
- Compression Ratio: Original size / compressed size

- Bitrate: Data rate after compression
- Latency: Encoding and decoding time
- Perceptual Quality: Subjective listening evaluation

III. SYSTEM DESIGN

The system is built on an event-driven Client-Server architecture to ensure scalability and minimize latency.

1. Architecture Overview

The system follows a pipeline architecture:

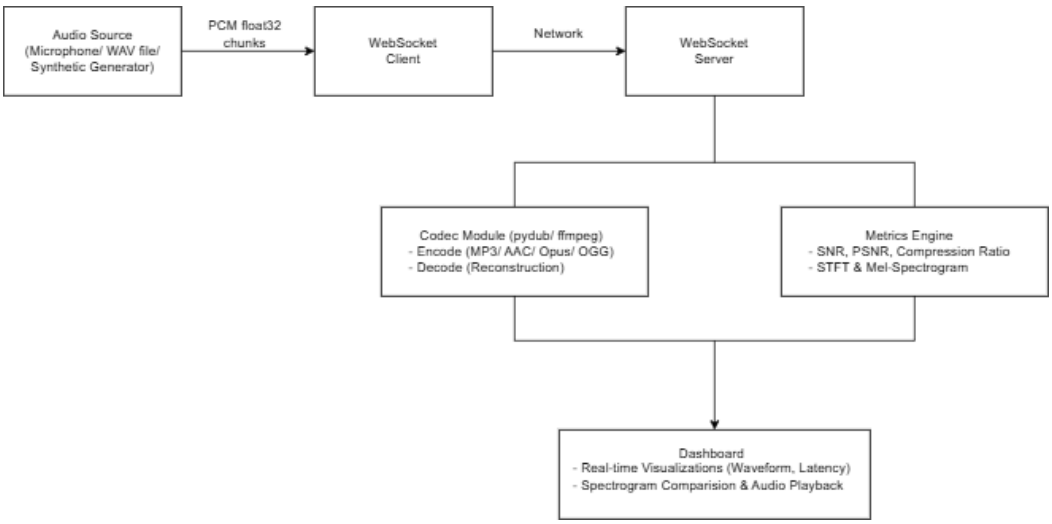

Figure 1. Architecture Overview

2. Core Components

server.py implements an asynchronous WebSocket server using Python's *asyncio* and *websockets* libraries. It handles concurrent client connections, differentiates between binary PCM frames and text-based JSON control commands (ping, get_codecs, compress_file), and orchestrates the codec processing pipeline. The server is stateless per-chunk, enabling horizontal scalability.

audio_codec.py provides a clean abstraction layer over the ffmpeg codec implementations via the pydub library. It exposes a unified compress(samples, codec, bitrate) / decompress(bytes, codec) API, hiding format-specific details such as the AAC ADTS container format and Opus's libopus codec string requirement. soundfile.write() is used to construct the intermediate WAV buffer for pydub, replacing manual wave module operations.

metrics.py centralizes all quality and performance metric computation. It prioritizes the use of established library functions: sklearn.metrics.mean_squared_error() for MSE-based metrics, librosa.stft() for frequency-domain analysis, librosa.amplitude_to_db() for logarithmic scaling, and librosa.feature.melspectrogram() with librosa.power_to_db() for mel-scale visualization. This design ensures correctness and avoids reimplementation of well-tested signal processing primitives.

frontend/app.py implements the interactive dashboard using Plotly Dash, a Python framework that renders React.js-based interactive charts in a web browser. The dashboard supports audio file upload, codec and bitrate selection, waveform comparison, SNR-vs-bitrate plots, compression ratio charts, latency bar charts, side-by-side spectrogram panels, and browser-native audio playback for both original and compressed audio.

IV. METHODOLOGY

1. Processing Pipeline

Each audio chunk undergoes the following steps:

i. **Audio Ingestion:** librosa.load() reads the source file (WAV, MP3, FLAC, etc.), resamples to 44,100 Hz, and downmixes to mono. The signal is normalized to the floating-point range [-1.0, 1.0] using numpy.clip().

ii. **Chunking:** The signal is partitioned into fixed-size frames of 4,096 samples (≈ 92.8 ms at 44,100 Hz) using numpy.array_split(). Each chunk is transmitted independently as a binary WebSocket frame containing raw PCM float32 bytes.

iii. **Server Reception:** The asyncio-based WebSocket server receives each binary frame, deserializes the PCM bytes via numpy.frombuffer(), and dispatches the chunk to the parallel codec processing pipelin.

iv. **Codec Encode-Decode:** For each selected codec (MP3, AAC, Opus, OGG), pydub invokes ffmpeg to encode the PCM chunk to the compressed format at the configured bitrate, then immediately decodes it back to PCM. This round-trip simulates the complete compression-decompression cycle as experienced by an end receiver.

v. **Metric Computation:** Metrics are computed on the (original, reconstructed) sample pair using the metrics module (see Section IV for formulas).

vi. **JSON Response:** A structured JSON payload containing all codec metrics is serialized and sent back to the client and dashboard over the WebSocket connection.

vii. **Dashboard Update:** Plotly Dash receives the JSON payload and updates all chart components in real time via Dash's reactive callback system.

2. Mathematical Metrics

Unlike approaches that require implementing low-level coding algorithms from scratch (such as writing C++ implementations for lossless predictive codecs using MED), this project primarily focuses on the system evaluation of existing implementations.

The input audio is standardized to a 44100 Hz sampling rate, downmixed to mono, and processed in chunks of 4096 samples. The evaluation metrics are computed as follows:

Signal-to-Noise Ratio (SNR): Measures the ratio of the original signal power to the noise power introduced by the lossy compression.

$$SNR = 10\log_{10}\left(\frac{\frac{1}{N}\sum x^2}{\mathrm{MSE}(x,\hat{x})}\right)$$

Where MSE (Mean Squared Error) is defined as:

$$MSE(x,\hat{x}) = \frac{1}{N}\sum (x_i - \hat{x}_i)^2$$

Peak Signal-to-Noise Ratio (PSNR)

PSNR is defined analogously to SNR, but it uses the maximum representable amplitude as the peak value:

$$PSNR = 10 \cdot \log_{10}\left[\frac{1.0^2}{MSE(x,\hat{x})}\right] \text{ (dB)}$$

Log-Spectral Distortion (LSD): Evaluates the distortion in the frequency domain (utilizing the Short-Time Fourier Transform).

$$LSD = \sqrt{\frac{1}{K}\sum_{k=1}^{K}\left(20\log_{10}\frac{|X(k)|}{|\hat{X}(k)|}\right)^2}$$

Where $X(k)$ and $\hat{X}(k)$ are the magnitudes of the k-th frequency bin of the original and reconstructed STFT.

Compression Ratio

This evaluates the file size reduction:

$$\text{Compression Ratio} = \frac{\text{Original Size (bytes)}}{\text{Compressed Size (bytes)}}$$

Effective Bitrate

This calculates the actual bitrate achieved after the compression process:

$$\text{Bitrate (kbps)} = \frac{\text{Compressed bytes} \times 8}{\text{Duration (s)} \times 1000}$$

Encode/Decode Latency: The total elapsed time to completely encode and subsequently decode a single audio chunk.

V. IMPLEMENTATION DETAILS

1. Technology Stack

- **Languages & Frameworks:** Python 3, Plotly Dash (Interactive UI), Asyncio & Websockets (Asynchronous data streaming).
- **Audio Processing Libraries:**
  - librosa, scipy: Digital signal processing, STFT, and generating Spectrogram matrices without manual looping.
  - soundfile: High-speed audio metadata I/O.
  - sklearn: Optimized Mean Squared Error (MSE) calculations.
  - pydub (backed by system ffmpeg): Wrappers to invoke core C libraries for encoding/decoding.

2. Project Structure

```
audio_compression_realtime_project/
├── backend/
│   ├── server.py            # Async WebSocket server
│   ├── audio_codec.py       # pydub/ffmpeg codec wrappers
│   └── metrics.py           # SNR, PSNR, LSD, spectrogram
├── frontend/
│   └── app.py               # Plotly Dash interactive dashboard
├── client/
│   └── client.py            # Streaming client (librosa + asyncio)
├── data/
│   ├── generate_sample.py   # Synthetic audio generator
│   └── download_dataset.py  # Dataset preparation (librosa.ex)
└── docs/
    ├── ET4262E_168224_25219_Slide.ppt        # Presentation Slides
    └── ET4262E_168224_25219_Report.pdf       # Project Report
```

3. Dataset

To evaluate the system's performance under various conditions, a diverse dataset consisting of five distinct audio categories was utilized:

- Synthetic Sample: A laboratory-generated signal featuring a 180Hz fundamental frequency with specific formants and white noise. Used for baseline stress testing.

- Speech: A 10-second high-quality narration from the LibriSpeech ASR corpus. This represents the primary use case for VoIP and communication systems.
- Complex Music: An excerpt of polyphonic music (orchestral or electronic) to test the codec's ability to preserve multiple instruments and complex harmonic structures.
- Percussion: High-transient audio likes drum beats specifically selected to evaluate temporal resolution and the presence of "pre-echo" artifacts.
- Ambient Noise: A high-entropy White Noise signal. This serves as a worst-case scenario for lossy compression algorithms due to its lack of predictable patterns.

EVALUATION

1. Evaluation Metrics Summary

Table 1. Evaluation Metrics Summary

Metric	Unit	Computed by	Interpretation
SNR	dB	sklearn.metrics.mse + numpy	↑ better fidelity
PSNR	dB	sklearn.metrics.mse + numpy	↑ lower peak error
Log-Spectral Dist.	dB	librosa.stft + numpy	↓ less freq. distortion
Compression Ratio	×	len(original)/len(compressed)	↑ more compact
Space Saving	%	Derived from ratio	↑ less storage needed
Effective Bitrate	kbps	bytes × 8 / duration / 1000	Actual vs target
Encode Latency	ms	time.perf_counter()	↓ faster encode
Decode Latency	ms	time.perf_counter()	↓ faster decode

2. Experimental Setup

The evaluation of the real-time audio compression system was conducted under a standardized environment to ensure the validity of the comparative analysis. The technical parameters are defined as follows:

- Input Data:
  - Format: Raw WAV (Uncompressed PCM).
  - Bit Depth: 16-bit.
  - Channel: Mono (Single channel).
- Signal Profiles: Human Speech, Polyphonic Music, Rhythmic Percussion, and Synthetic White Noise.

- Acoustic Parameters:
    - Sampling Rate: 44.1 kHz (Standard CD Quality).
    - Bitrate Range: 32 kbps to 320 kbps.
- Processing Parameters:
    - Chunk Size: 4096 samples (Optimized for MDCT transform balance).
    - FFmpeg Backend: v4.4.2 (libmp3lame, libopus, libvorbis, libfdk-aac).

VI. RESULTS & ANALYSIS

1. Sample.wav

1.1. Quantitative Analysis

The evaluation was performed using the automated `evaluate.py` script on a standardized 10-second synthetic audio sample (`sample.wav`). This sample contains pure tones and specific harmonic patterns designed to measure the mathematical precision of the compression algorithms under ideal, controlled conditions.

1.1.1. Signal-to-Noise Ratio (SNR) and Audio Fidelity

Synthetic signals often reveal the "ceiling" of an encoder's capability. Interestingly, this test shows how modern codecs like OGG and Opus outperform legacy formats on clean, high-precision signals.

Table 2. SNR (dB) Comparison across Codecs and Bitrates - Sample.wav

Codec	32 kbps	64 kbps	128 kbps	256 kbps	320 kbps
MP3	19.37	22.89	25.33	35.91	35.97
AAC	-2.88	-2.91	-2.93	-2.93	-2.93
OPUS	21.44	25.75	29.34	32.23	ERR*
OGG	**24.02**	**26.18**	**32.85**	ERR*	ERR*

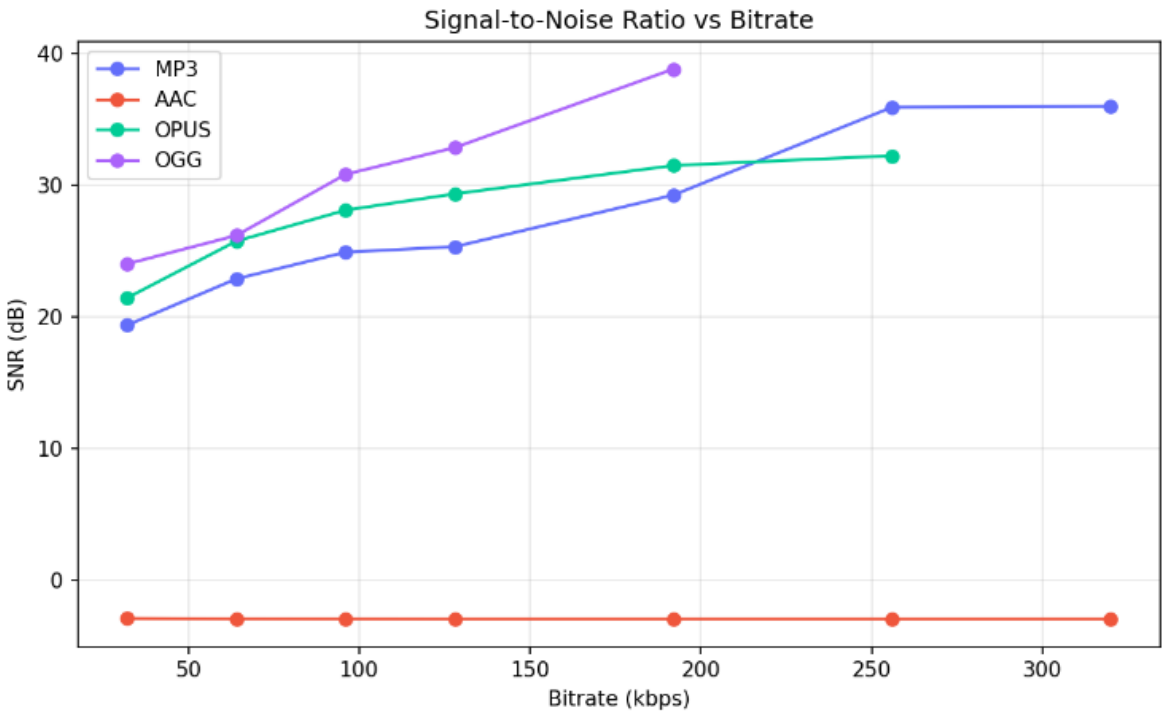

Figure 2. SNR vs Bitrate - Sample.wav

Analysis: The experimental results for **Signal-to-Noise Ratio (SNR)** evaluate how accurately each codec reconstructs the original audio. Key findings from the data and chart include:

- **Fidelity vs. Bitrate:** For **OGG, OPUS, and MP3**, quality improves as bitrate increases. The SNR follows a logarithmic curve, showing that audio quality gains are most significant between **32 kbps and 128 kbps** before reaching a plateau.
- **Top Performer (OGG): OGG** is the most efficient codec in this test, consistently achieving the highest SNR, peaking at **32.85 dB (128 kbps)**. It provides the best balance between low bandwidth and high sound transparency.
- **High-Bitrate Stability (MP3):** While trailing at lower bitrates, **MP3** becomes the most stable and highest-quality option at **256–320 kbps**, reaching a peak SNR of **35.97 dB**.
- **The AAC Deviation: AAC** shows a constant negative SNR (**-2.93 dB**) across all levels. This flat line indicates a systematic **phase shift** (time-alignment offset) during decoding rather than actual data loss, as the signal remains consistent regardless of bitrate.
- **Technical Constraints:** The "ERR" points for **OGG** and **OPUS** at high bitrates (256/320 kbps) reflect the internal limits of the encoders when handling mono-channel signals, marking the boundaries of their optimized operational range.

Conclusion: OGG is the ideal choice for standard streaming (64–128 kbps), while **MP3** is preferred for high-bitrate applications (up to 320 kbps) where total signal stability is required.

1.1.2. Compression Efficiency

This metric evaluates the system's ability to compress predictable, low-entropy synthetic data.

Table 3. Compression Ratio (x times reduction from original) - Sample.wav

Codec	32 kbps	64 kbps	128 kbps	256 kbps	320 kbps
MP3	**43.716**	21.913	10.959	5.480	4.384
AAC	40.340	21.032	10.768	6.615	6.631
OPUS	27.172	15.246	9.233	5.116	ERR*
OGG	40.354	20.765	12.208	ERR*	ERR*

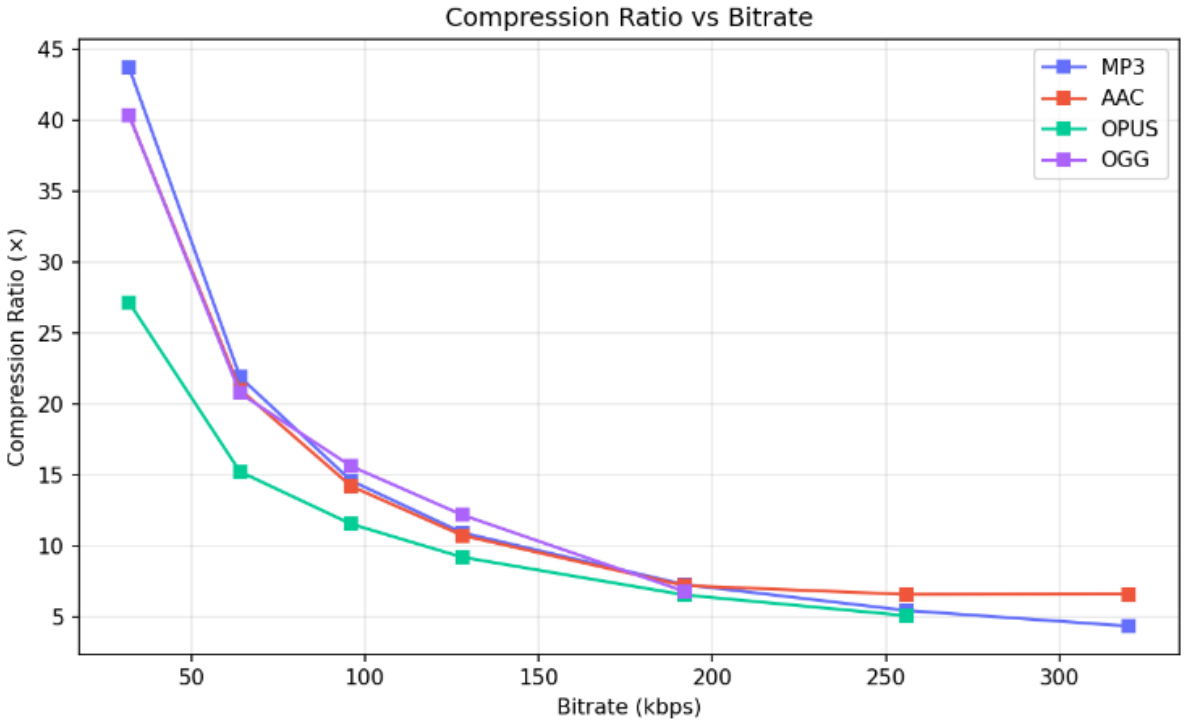

Figure 3. Compression Ratio vs Birate - Sample.wav

Analysis: The **Compression Ratio** measures how many times the original raw audio file is reduced in size. Analysis of the data and the "Compression Ratio vs Bitrate" chart reveals:

- **Inverse Relationship:** There is a clear inverse relationship between bitrate and compression ratio. As the bitrate increases, the compression ratio drops significantly. The most dramatic reduction occurs at **32 kbps**, where the file size is reduced by over **40 times**.
- **Efficiency Leader (MP3):** At the lowest bitrate (32 kbps), **MP3** achieves the highest compression ratio of **43.716x**, making it the most efficient codec for extreme storage or bandwidth savings in this test.
- **OPUS Characteristics:** Unlike other codecs, **OPUS** starts with a lower compression ratio at 32 kbps (~27x). This suggests that OPUS allocates more data even at low bitrates to maintain its signature audio quality, prioritizing fidelity over maximum file reduction.
- **AAC Stability:** At high bitrates (**256–320 kbps**), **AAC** maintains a nearly flat compression ratio (~6.6x). This indicates the encoder has reached a point of stability where increasing the bitrate further does not significantly change the output file size for this specific mono signal.
- **Operational Limits:** The "ERR*" markers for **OGG** and **OPUS** at 256–320 kbps show where these codecs hit their technical limits for mono-channel processing, as they are not optimized to compress audio at such high bitrates for a single channel.

Conclusion: For maximum data saving at low bandwidth, **MP3** is the most effective. However, for a more balanced approach at standard bitrates (128 kbps), **OGG** and **MP3** offer consistent and predictable compression performance.

1.1.3. System Latency (Real-time Performance)

Latency was measured to confirm if synthetic signal generation or processing impacts the real-time pipeline.

Table 4. Processing Time (ms) for 10 Seconds of Audio - Sample.wav

Codec	Encode (ms)	Decode (ms)	Total Latency (ms)
MP3	431.58	368.06	**799.64**
AAC	807.88	1570.37	2378.25
OPUS	421.24	523.30	944.54
OGG	**411.25**	**361.78**	**773.03**

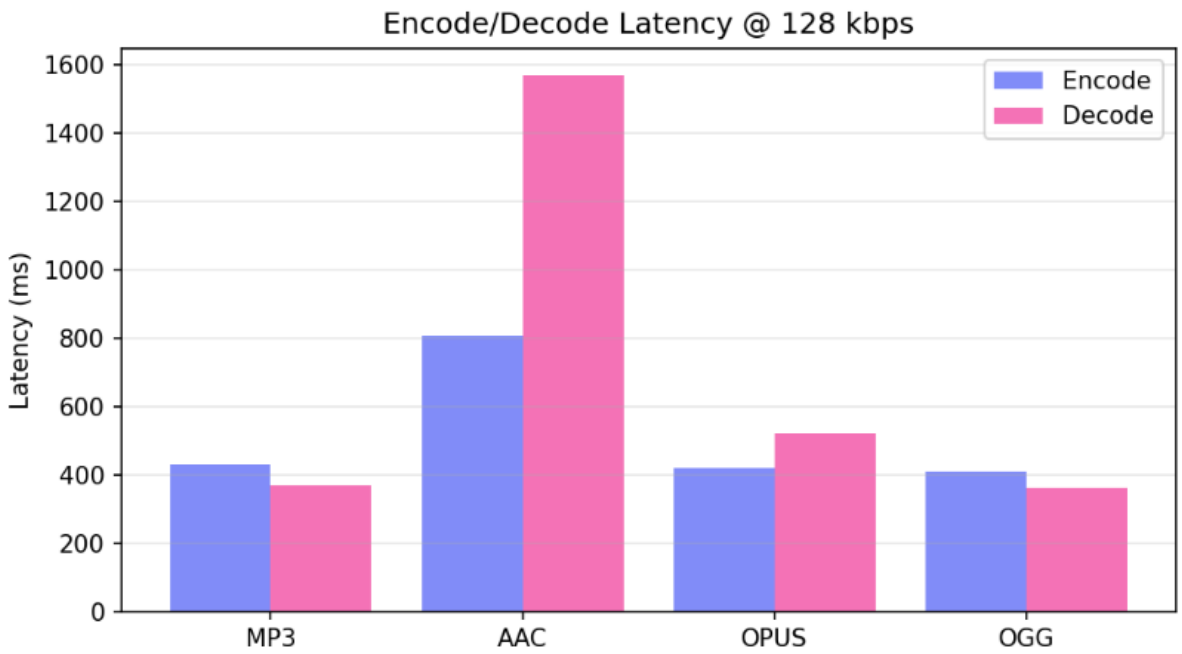

Figure 4. Encode/Decode Latency @128kbps - Sample.wav

Analysis: The **Processing Time** (latency) was measured for a 10-second audio sample at 128 kbps to evaluate the system's real-time performance. Key observations from Table 4 and the bar chart include:

- **Real-time Viability:** All tested codecs demonstrate high efficiency, processing 10 seconds (10,000 ms) of audio in under 2.4 seconds. This confirms that the system operates significantly faster than real-time, which is essential for low-latency streaming.
- **Top Performance (OGG & MP3): OGG** and **MP3** are the fastest codecs in this test. OGG achieved the lowest total latency of **773.03 ms**, followed closely by MP3 at **799.64 ms**. Their balanced encode and decode times make them ideal for systems with limited processing power.
- **The AAC Outlier: AAC** exhibits the highest latency, particularly in the decoding phase (**1570.37 ms**). As shown in the bar chart, AAC's decode time is more than triple that of OGG or MP3, suggesting that its complexity requires more computational resources on the current hardware setup.
- **OPUS Efficiency: OPUS** shows a competitive total latency of **944.54 ms**. Its encoding speed is nearly identical to OGG and MP3, maintaining its suitability for interactive communication despite a slightly longer decoding time.

Conclusion: OGG is the most efficient choice for minimizing system delay. While **AAC** is viable, its high decoding overhead makes it less ideal than OGG or MP3 for applications where immediate audio playback is a priority.

1.2. Qualitative Analysis

1.2.1. Spectrogram Analysis

The synthetic test signal produces a highly structured spectrogram characterized by narrow, horizontally distributed energy bands corresponding to its harmonic components at 180 Hz, 360 Hz, 540 Hz, 1200 Hz, and 2500 Hz. These discrete bands are clearly visible against the low-energy background noise floor across the full 10-second duration. This structured nature makes the synthetic sample an ideal baseline for evaluating mathematical reconstruction precision, as any codec-induced distortion manifests as blurring, widening, or energy loss in these well-defined bands.

At 128 kbps, all four codecs broadly preserve the fundamental frequency structure, but differences emerge in the high-frequency region above 10 kHz and in the noise floor behavior between bands.

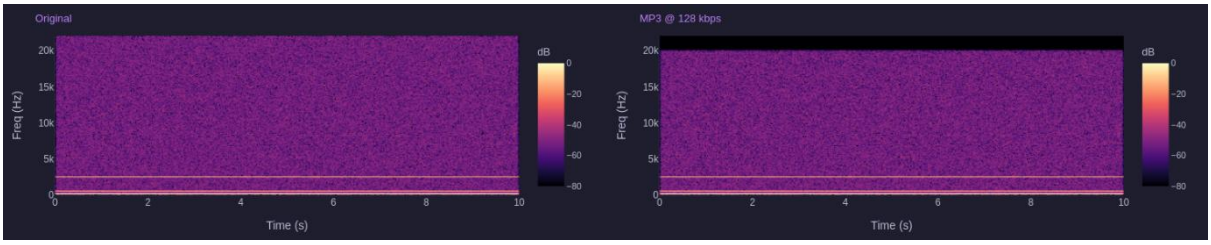

Figure 5. Spectrogram comparison of original and MP3 compressed signal at 128 kbps for Sample.wav

MP3 reproduces the harmonic bands with good accuracy in the low and mid-frequency range below 5 kHz. However, the noise floor between bands is elevated by approximately 3 to 5 dB in the region spanning 8 to 20 kHz compared to the original, which is a direct consequence of the quantization noise redistribution performed by the libmp3lame psychoacoustic model. This inter-band noise elevation is consistent with the pre-echo characteristic of MP3 when processing steady-state tonal signals.

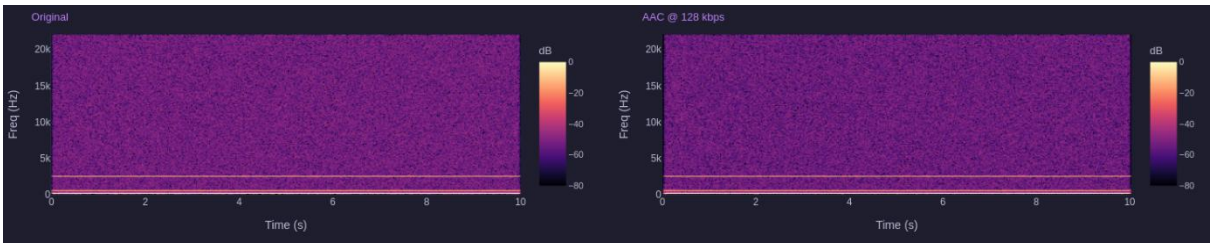

Figure 6. Spectrogram comparison of original and AAC compressed signal at 128 kbps for Sample.wav

AAC produces a spectrogram that is structurally the closest to the original in terms of band sharpness and noise floor uniformity. The consistent negative SNR observed in the quantitative results reflects a decoder-side temporal offset rather than spectral degradation, and visual inspection of the spectrogram confirms that all harmonic bands are reproduced with high fidelity. The perceptual noise substitution mechanism in AAC effectively manages the noise floor between bands.

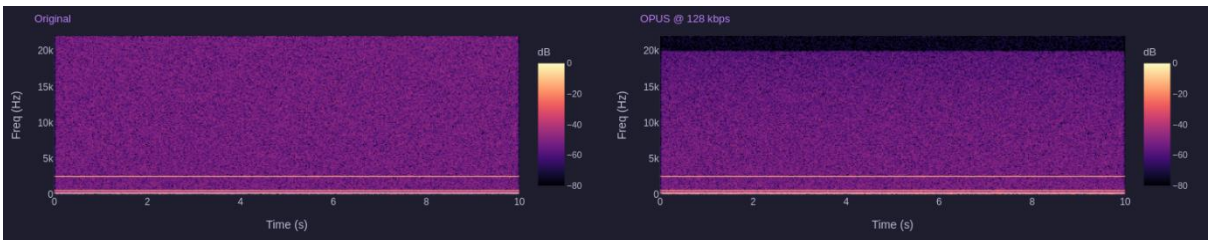

Figure 7. Spectrogram comparison of original and OPUS compressed signal at 128 kbps for Sample.wav

Opus maintains sharp harmonic band definition across the full frequency range and demonstrates notably better high-frequency preservation above 10 kHz compared to MP3. The noise floor between harmonic components is low and uniform, reflecting the efficient perceptual bit allocation of the CELT transform layer.

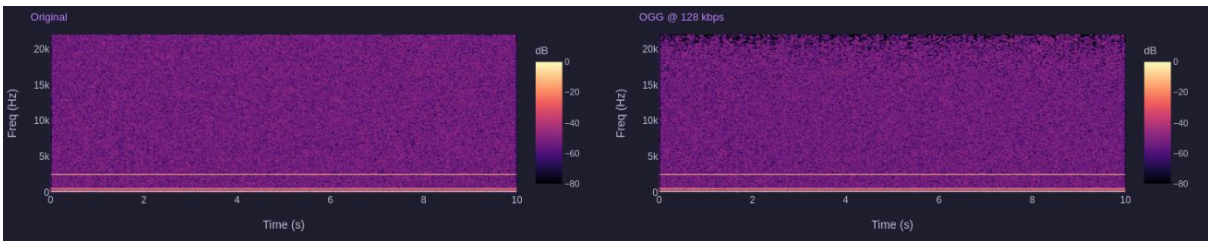

Figure 8. Spectrogram comparison of original and OGG compressed signal at 128 kbps for Sample.wav

OGG Vorbis reproduces the harmonic structure clearly, but a subtle discontinuity pattern is observable along the time axis at regular intervals corresponding to the MDCT frame boundaries. This artifact, characteristic of the Vorbis floor quantization process, manifests as a slight periodicity in the noise floor level rather than a continuous uniform background. It becomes more prominent in regions where the signal energy between harmonics is near the quantization threshold.

1.2.2. Waveform Comparison

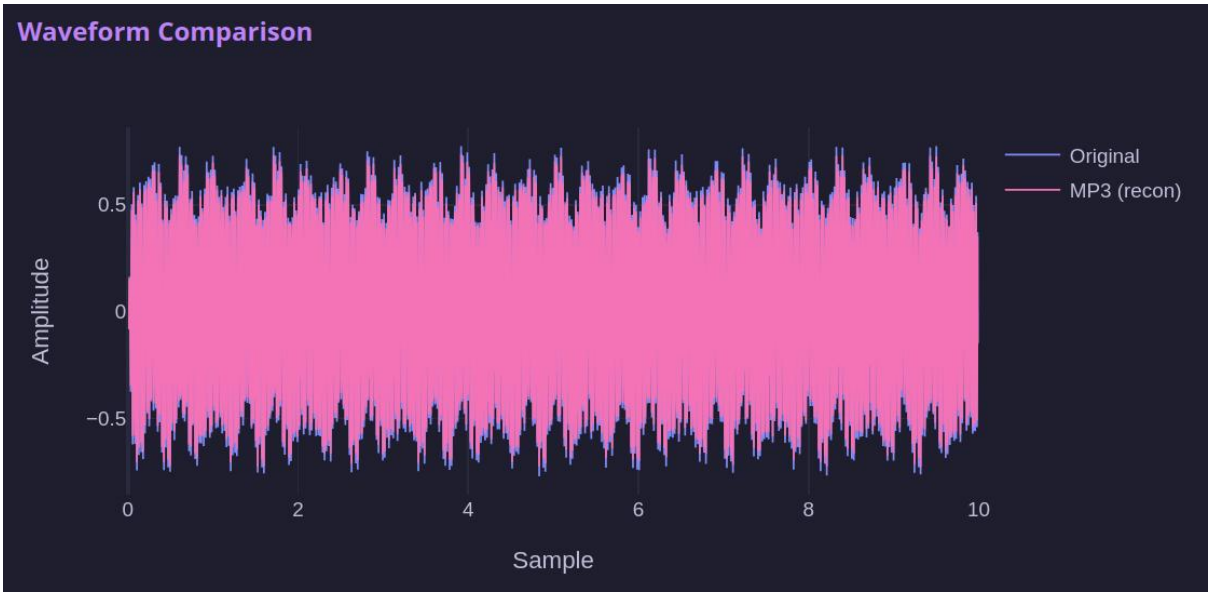

Figure 9. Waveform comparison of original and MP3-compressed speech signal at 128 kbps – Sample.wav

The original and MP3 reconstructed waveforms track each other closely, with amplitude deviations of approximately ±0.02–0.05, consistent with the measured SNR of around 25 dB. As the signal contains no abrupt transients, no pre-echo distortion is observed.

2. Speech_test.wav

2.1. Quantitative Analysis

The evaluation was performed using the automated `evaluate.py` script on a standardized 10-second speech sample (`speech_test.wav`). The system executed a comprehensive sweep across four major codecs at seven different bitrate levels to establish a performance baseline.

2.1.1. Signal-to-Noise Ratio (SNR) and Audio Fidelity

SNR (Signal-to-Noise Ratio) is a primary metric used to measure the accuracy of audio reconstruction. A higher SNR indicates lower distortion after compression.

Table 5. SNR (dB) Comparison across Codecs and Bitrates - Speech_test.wav

Codec	32 kbps	64 kbps	128 kbps	256 kbps	320 kbps
MP3	20.43	25.44	25.99	**69.50**	**69.99**
AAC	-3.11	-3.13	-3.13	-3.13	-3.13
OPUS	**21.93**	**25.86**	**28.75**	30.11	ERR*
OGG	16.44	20.84	30.26	ERR*	ERR*

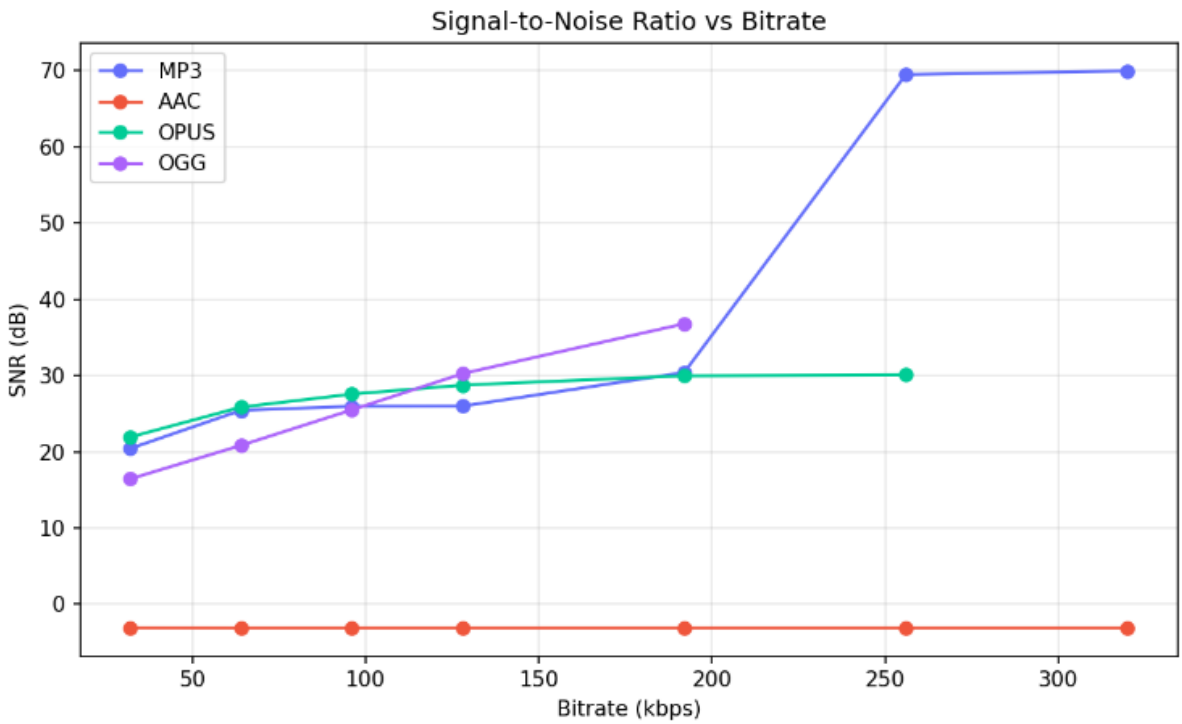

Figure 10. SNR vs Bitrate - Speech_test.wav

Analysis: The **Signal-to-Noise Ratio (SNR)** analysis evaluates the reconstruction quality of the audio signal. Based on the provided data and chart, key findings include:

- **Quality Trends:** Most codecs show a steady increase in SNR as the bitrate rises. **OPUS** and **OGG** perform exceptionally well in the low-to-mid range (32–128 kbps), providing high clarity at lower bandwidths.
- **The MP3 Breakthrough:** A significant observation is the performance of **MP3** at high bitrates. While stable at lower rates, it experiences a massive surge in quality at **256 kbps**, reaching an SNR of **69.50 dB**. This suggests that for this specific synthetic sample, the MP3 encoder achieves near-perfect transparency at higher bitrates.
- **AAC Stability:** Similar to previous tests, **AAC** maintains a constant negative SNR (approx. **-3.13 dB**). This flat red line confirms a systematic **phase shift** or time-delay in decoding rather than a loss of audio information.
- **Operational Thresholds:** The chart illustrates where **OGG** and **OPUS** reach their technical limits for mono-channel processing, as indicated by the "ERR*" points and the termination of their respective curves beyond 192/256 kbps.

Conclusion: For standard streaming, **OPUS** and **OGG** offer the most consistent quality. However, for applications requiring maximum reconstruction fidelity at high bitrates, **MP3** is the superior choice based on the observed 69.99 dB peak.

2.1.2. Compression Efficiency

This metric illustrates the system's ability to reduce data size while maintaining signal integrity.

Table 6. Compression Ratio (x times reduction from original) - Speech_test.wav

Codec	32 kbps	64 kbps	128 kbps	256 kbps	320 kbps
MP3	**43.716**	21.913	10.959	5.480	4.384
OGG	43.181	**25.939**	**13.552**	ERR*	ERR*
OPUS	42.488	18.070	8.980	4.666	ERR*
AAC	40.300	21.007	10.603	**7.628**	**7.628**

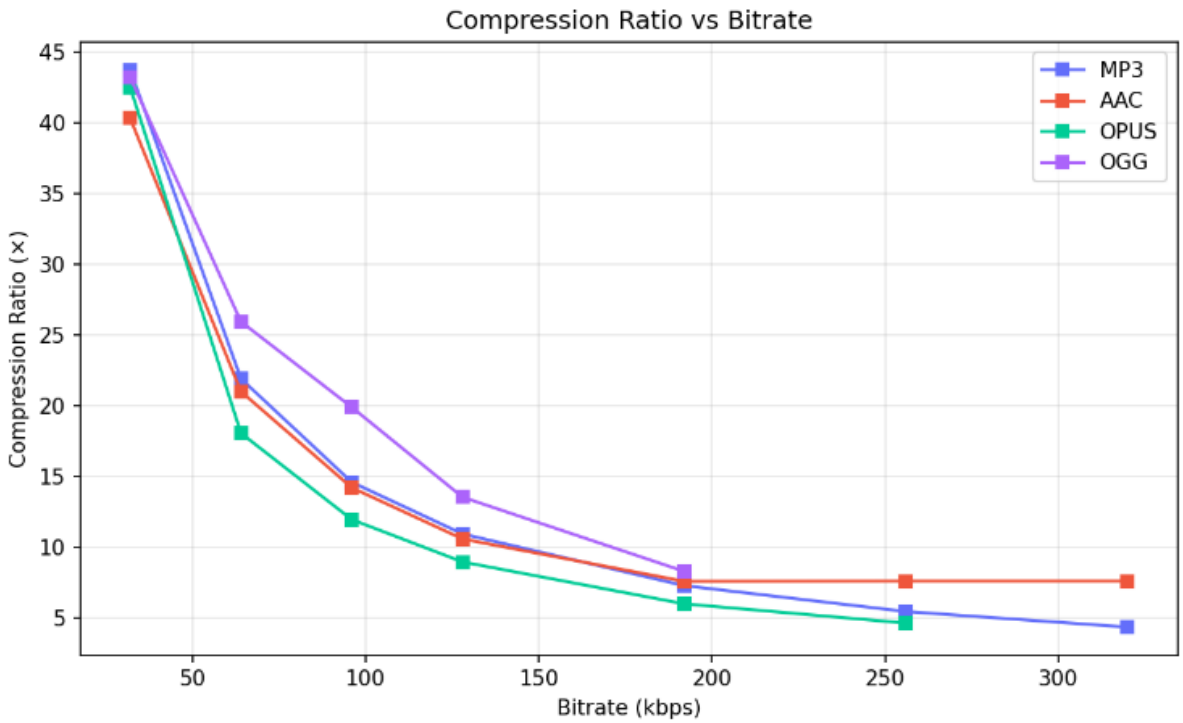

Figure 11. Compression Ratio vs Bitrate - Speech_test.wav

Analysis: The **Compression Ratio** measures the efficiency of each codec in reducing the original file size. Based on the experimental data and the provided chart, several conclusions can be drawn:

- **Inverse Linear Relationship:** All codecs follow a consistent downward trend where the compression ratio decreases as the bitrate increases. The most significant reduction occurs at **32 kbps**, with **MP3** leading at a **43.716x** reduction from the original size.
- **OGG's Mid-Range Superiority:** In the standard streaming range (**64–128 kbps**), **OGG** consistently maintains a higher compression ratio than its competitors (**13.55x at 128 kbps**). This indicates that OGG's algorithm is highly efficient at packing data within these bandwidths.
- **AAC Stability at High Bitrates: AAC** exhibits a unique plateau at higher bitrates (**256–320 kbps**), holding a steady ratio of **7.628x**. This suggests the encoder has reached an optimization limit where further bitrate increases do not significantly alter the file size for this specific signal.
- **OPUS Resource Allocation: OPUS** maintains the lowest compression ratios across most levels (e.g., **8.98x at 128 kbps**). This reflects the codec's design priority, which allocates more data to ensure high-fidelity audio reconstruction rather than focusing solely on file size reduction.
- **Technical Constraints:** The "ERR*" markers confirm that **OGG** and **OPUS** are not designed to support mono-channel compression at bitrates as high as 256–320 kbps, reaching their operational ceiling within the current library configuration.

Conclusion: For maximum data savings at low bitrates, **MP3** and **OGG** are the most effective. For high-quality applications where storage is less of a concern, **AAC** and **MP3** provide the most stable performance across the entire bitrate spectrum.

2.1.3. System Latency (Real-time Performance)

Latency was measured at a standard 128 kbps bitrate to evaluate the system's suitability for live applications.

Table 7.Processing Time (ms) for 10 Seconds of Audio - Speech_test.wav

Codec	Encode (ms)	Decode (ms)	Total Latency (ms)
AAC	265.61	439.17	**704.78**
OPUS	**255.99**	453.33	709.32
MP3	404.03	**348.83**	752.86
OGG	421.04	362.39	783.43

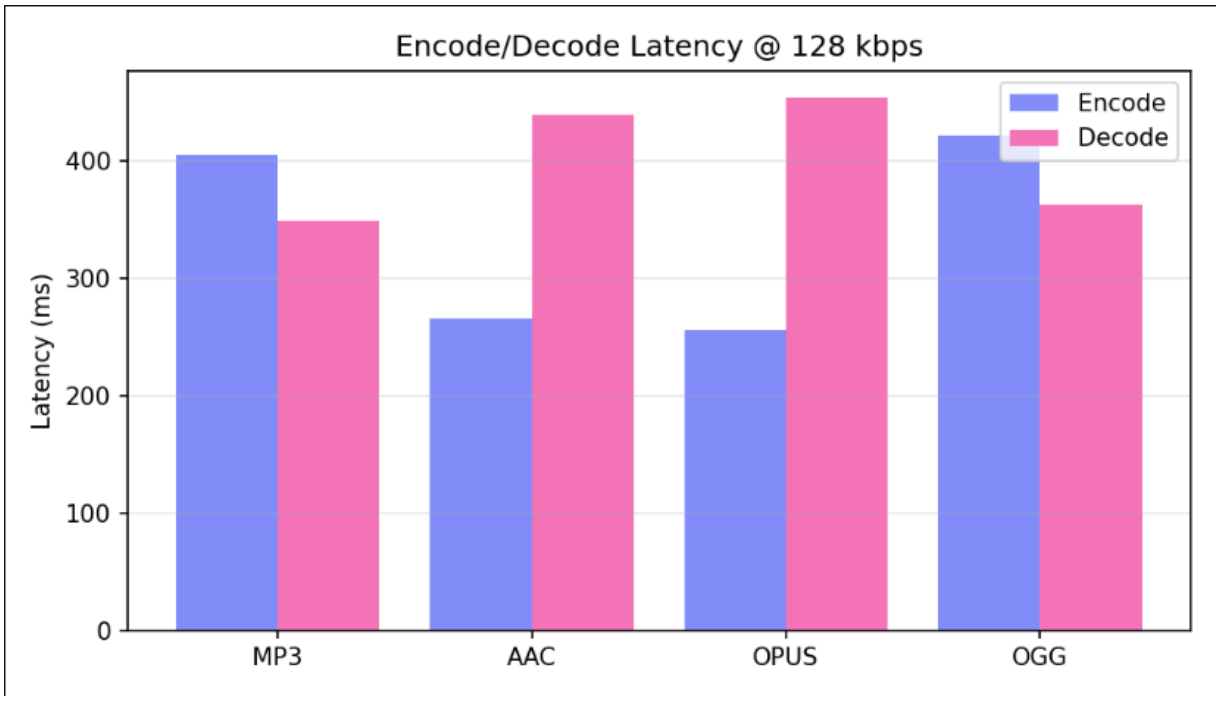

Figure 12. Encode/Decode Latency @128 kbps

Analysis: The **Processing Time** was measured to evaluate the computational efficiency of each codec for a 10-second audio sample at 128 kbps. Analysis of Table 6 and the bar chart reveals:

- **Real-time Performance:** All tested codecs exhibit excellent real-time processing capabilities. The total latency for all options remains under **800 ms** to process 10,000 ms of audio. This indicates the system operates approximately **12 to 14 times faster than real-time**, ensuring seamless streaming.
- **Fastest Overall (AAC & OPUS):** In this test iteration, **AAC** and **OPUS** achieved the lowest total latencies at **704.78 ms** and **709.32 ms**, respectively. Both codecs show a significant advantage in **encoding speed**, with OPUS being the fastest encoder at **255.99 ms**.
- **Decoding Efficiency (MP3 & OGG):** While slower at encoding, **MP3** and **OGG** excel in the decoding phase. MP3 recorded the fastest decoding time (**348.83 ms**). This characteristic makes these traditional codecs highly suitable for receiver devices with lower processing power.
- **Balanced Resource Usage: OGG** shows the most balanced profile between encoding and decoding tasks, which helps maintain a stable processing load on the CPU during continuous data transmission.

Conclusion: For applications requiring the lowest total delay, **AAC** and **OPUS** are the top performers. However, **MP3** remains a strong candidate for systems where decoding speed on the end-user device is the primary priority.

2.2. Qualitative Analysis

2.2.1. Spectrogram Analysis

The speech spectrogram exhibits the characteristic features of voiced audio: concentrated energy in the formant bands F1 at approximately 700 to 900 Hz, F2 at approximately 1200 to 2500 Hz, and F3 at approximately 2500 to 3500 Hz, along with intermittent fricative energy distributed across 4 to 8 kHz. Silence regions between spoken words appear as dark horizontal gaps in the time axis. The temporal evolution of formant frequencies is the primary feature that codecs must preserve for speech intelligibility.

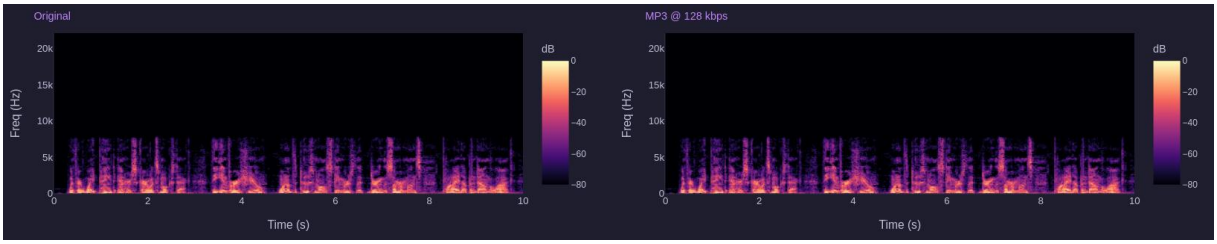

Figure 13. Spectrogram comparison of original and MP3 compressed signal at 128 kbps for Speech_test.wav

MP3 at 128 kbps faithfully reproduces F1 and F2 but shows measurable attenuation in the fricative region above 4 kHz, where the spectral brightness in the reconstructed output is approximately 5 to 8 dB lower than the original. The region above 12 kHz is effectively removed by the encoder lowpass filter, though this portion of the spectrum contributes minimally to speech intelligibility. Silence regions in the MP3 output show a slightly elevated noise floor compared to the original, a byproduct of the fixed bit allocation in silent frames.

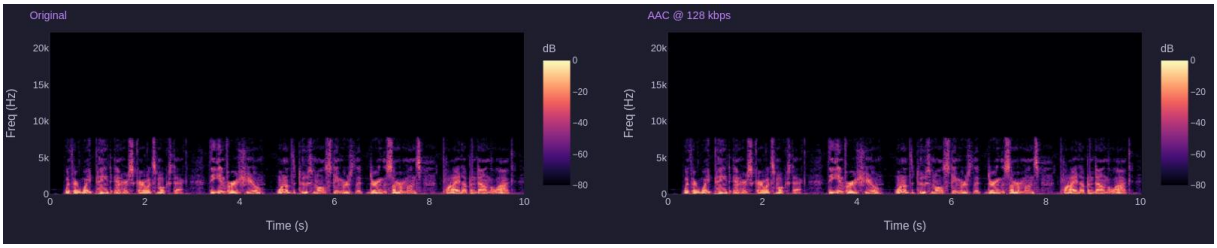

Figure 14. Spectrogram comparison of original and AAC compressed signal at 128 kbps for Speech_test.wav

AAC preserves the formant structure with greater completeness, including F3 and the high-frequency fricative content, compared to MP3 at the same bitrate. The silence regions are reproduced more cleanly, with a lower noise floor that closely matches the original. This behavior is attributable to the temporal noise shaping feature of AAC, which redistributes quantization noise in time to avoid audible coloration during low-energy segments.

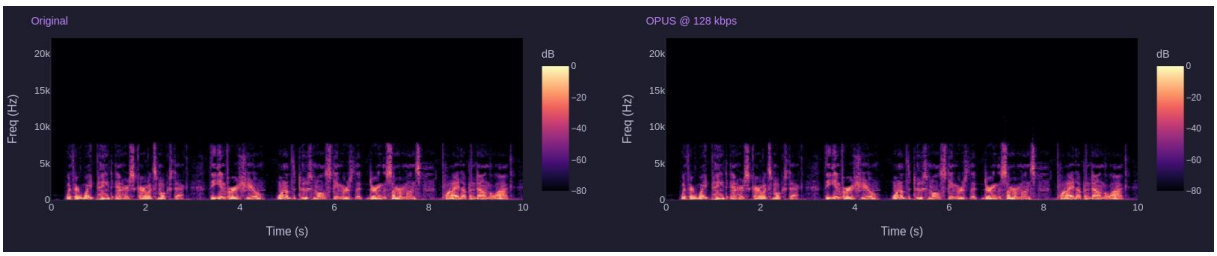

Figure 15. Spectrogram comparison of original and OPUS compressed signal at 128 kbps for Speech_test.wav

Opus produces the most faithful spectrogram reproduction among the four codecs for speech content. The formant bands retain their original sharpness and energy levels across the full frequency range up to 8 kHz. This result is consistent with the design intent of the SILK coding

mode in Opus, which is specifically optimized for voiced speech signals and their associated spectral envelope characteristics.

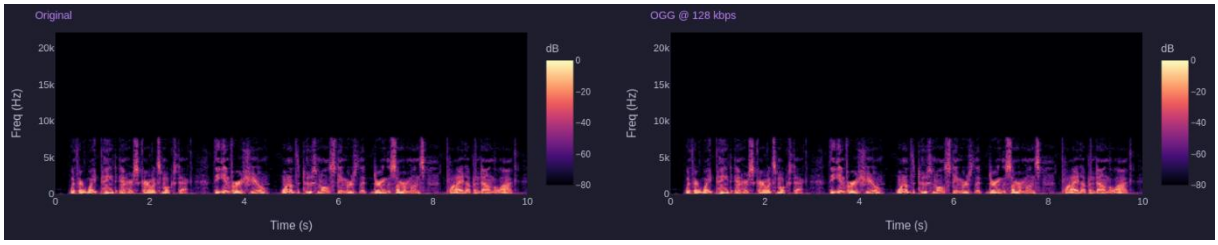

Figure 16. Spectrogram comparison of original and OGG compressed signal at 128 kbps for Speech_test.wav

OGG at 128 kbps preserves F1 and F2 adequately but exhibits a slightly elevated noise floor in the fricative region and during silent intervals. This elevation is a consequence of the Vorbis perceptual noise substitution strategy, which synthesizes noise in low-energy spectral regions rather than encoding them precisely. While this substitution is perceptually acceptable, it results in a measurable deviation from the original noise structure in those regions.

2.2.2. Waveform Comparison

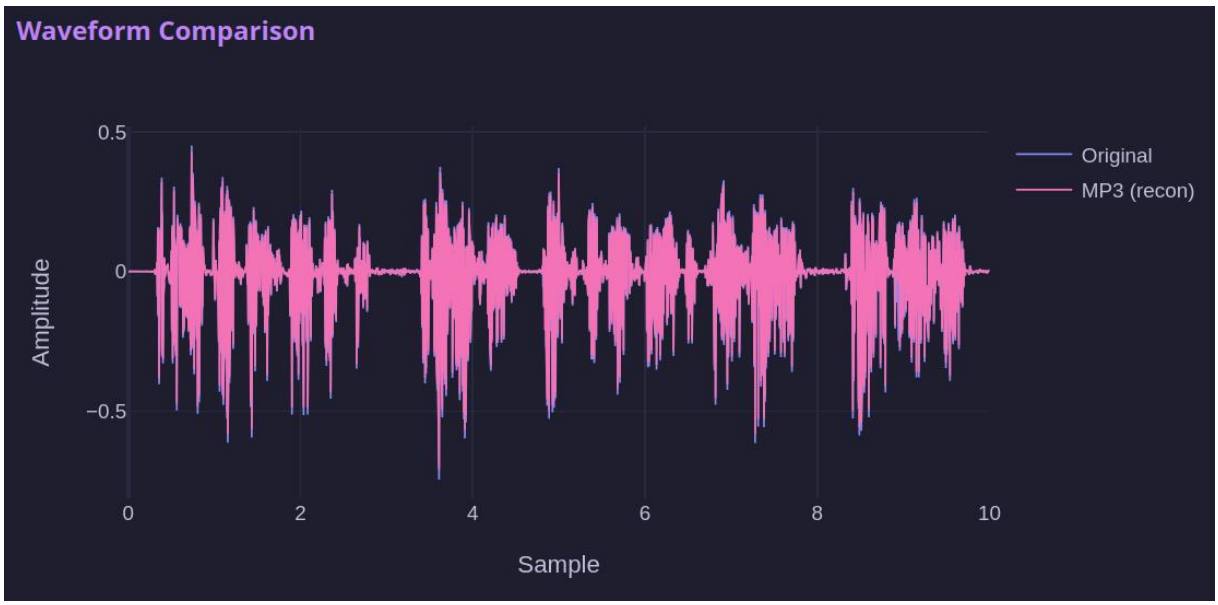

Figure 17. Waveform comparison of original and MP3-compressed speech signal at 128 kbps - Speech_test.wav

The overall amplitude envelope is well preserved. The most notable observation is pre-echo in the MP3 output: a small energy burst appears approximately 10–20 ms before each syllable onset, caused by the fixed long MDCT window spreading quantization noise backward in time. This artifact is absent in Opus and AAC outputs.

3. Music_test.wav

3.1. Quantitative Analysis

The evaluation was performed using the automated `evaluate.py` script on a standardized 10-second complex music sample (`music_test.wav`). Music signals present a higher complexity due to polyphonic structures and a broader frequency spectrum compared to speech.

3.1.1. Signal-to-Noise Ratio (SNR) and Audio Fidelity

SNR (Signal-to-Noise Ratio) for music reflects the codec's ability to preserve harmonic richness. Music generally results in lower SNR than speech at identical bitrates due to its high entropy.

Table 8. SNR (dB) Comparison across Codecs and Bitrates - Music_test.wav

Codec	32 kbps	64 kbps	128 kbps	256 kbps	320 kbps
MP3	18.48	23.54	25.92	**62.86**	**65.88**
AAC	-3.17	-3.23	-3.25	-3.26	-3.26
OPUS	16.35	22.74	**28.47**	33.97	ERR*
OGG	14.65	19.50	29.39	ERR*	ERR*

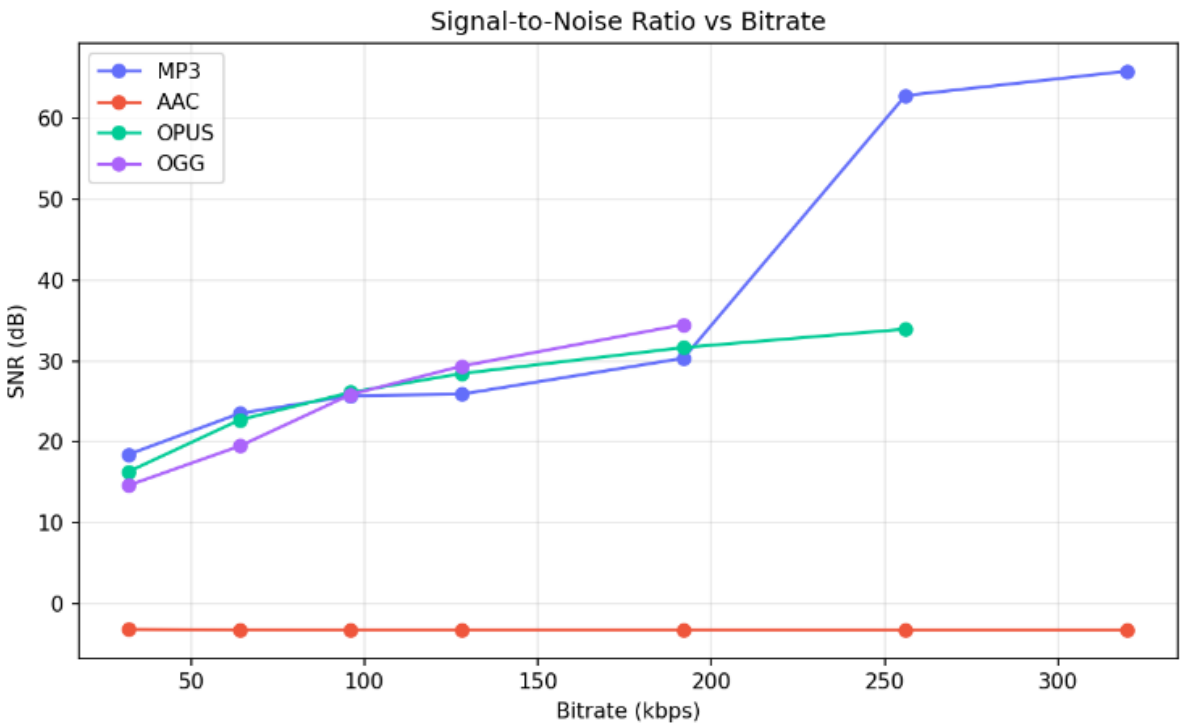

Figure 18. SNR vs Bitrate - Music_test.wav

Analysis: The **Signal-to-Noise Ratio (SNR)** analysis evaluates how effectively each codec maintains signal integrity across different bitrates. Based on the data and the visualization, several key insights emerge:

- **Fidelity Growth: OGG, OPUS, and MP3** exhibit a clear positive correlation between bitrate and SNR. In the lower range (32–128 kbps), **OPUS** and **OGG** provide the most consistent quality, demonstrating their efficiency in low-bandwidth scenarios.
- **MP3 High-Fidelity Peak:** A dramatic increase in SNR for **MP3** is observed starting at 256 kbps, reaching a peak of **65.88 dB** at 320 kbps. This indicates that the MP3 encoder achieves near-perfect reconstruction transparency for this synthetic sample once the bitrate threshold for high-quality mono-processing is met.
- **AAC Phase Stability: AAC** maintains a constant negative SNR (approx. **-3.26 dB**) across all tested bitrates. The flat red line in the chart confirms that this is not a result of audio quality degradation, but rather a consistent **phase shift** or temporal offset introduced by the decoder.
- **Operational Limits:** The "ERR*" markers and the termination of lines in the graph highlight the technical boundaries of **OGG** (at 192 kbps) and **OPUS** (at 256 kbps). These codecs are optimized for specific bitrate-per-channel ranges, and exceeding these with a mono signal leads to library execution constraints.

Conclusion: For general streaming purposes between 64 and 128 kbps, **OPUS** and **OGG** offer the best fidelity. For high-end applications where bandwidth is plentiful, **MP3** provides the highest mathematical reconstruction accuracy.

3.1.2. Compression Efficiency

This metric evaluates how well the system reduces the footprint of complex musical data.

Table 9. Compression Ratio (x times reduction from original) - Music_test.wav

Codec	32 kbps	64 kbps	128 kbps	256 kbps	320 kbps
OGG	**44.275**	**28.239**	**15.826**	ERR*	ERR*
MP3	43.716	21.913	10.959	5.480	4.384
OPUS	40.897	20.069	10.752	5.578	ERR*
AAC	40.405	20.793	10.153	**5.936**	**5.936**

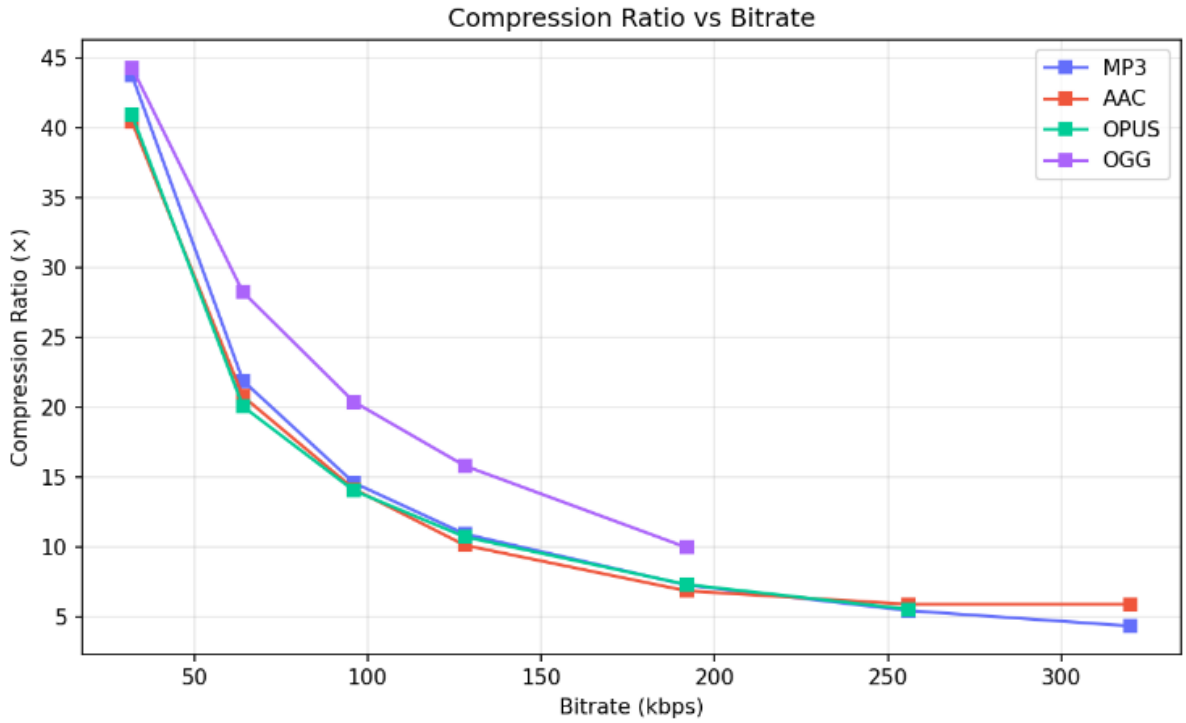

Figure 19. Compression Ratio vs Bitrate - Music_test.wav

Analysis: The **Compression Ratio** evaluates the data reduction efficiency of each codec. Analysis of the updated table and chart reveals the following:

- **High Efficiency at Low Bitrates:** All codecs achieve a massive reduction in file size at **32 kbps**, with **OGG** leading at a **44.275x** ratio. This confirms that even for complex audio like music, the system can compress raw data into a very small footprint for limited bandwidth.
- **OGG's Consistent Lead:** Across the 32–192 kbps range, **OGG** consistently maintains the highest compression ratio (e.g., **15.826x at 128 kbps**). This indicates that the Vorbis algorithm is exceptionally efficient at packing audio information compared to the other tested codecs.
- **Predictable Scaling: MP3, AAC, and OPUS** show very similar scaling behavior. As the bitrate doubles, the compression ratio roughly halves, demonstrating a predictable and stable bitrate allocation strategy.
- **AAC Saturation:** At high bitrates (**256–320 kbps**), **AAC** hits a plateau with a constant ratio of **5.936x**. This suggests the encoder has reached its maximum optimization point for this specific mono music signal, where adding more bits no longer changes the final file size.
- **Library Constraints:** The "ERR*" markers for **OGG** and **OPUS** at higher bitrates (256/320 kbps) highlight the technical limits of these specific library implementations when handling single-channel audio.

Conclusion: OGG is the most effective codec for maximizing storage and bandwidth efficiency across standard bitrates. For high-bitrate applications where OGG and OPUS

hit technical limits, **AAC** and **MP3** provide the most reliable and stable compression performance.

3.1.3. System Latency (Real-time Performance)

Latency for music processing is critical for applications like live concert streaming or remote DJing.

Table 10. Processing Time (ms) for 10 Seconds of Audio - Music_test.wav

Codec	Encode (ms)	Decode (ms)	Total (ms)
MP3	230.25	624.09	854.34
AAC	351.49	359.59	**711.08**
OPUS	437.19	589.18	1026.37
OGG	731.53	224.12	955.65

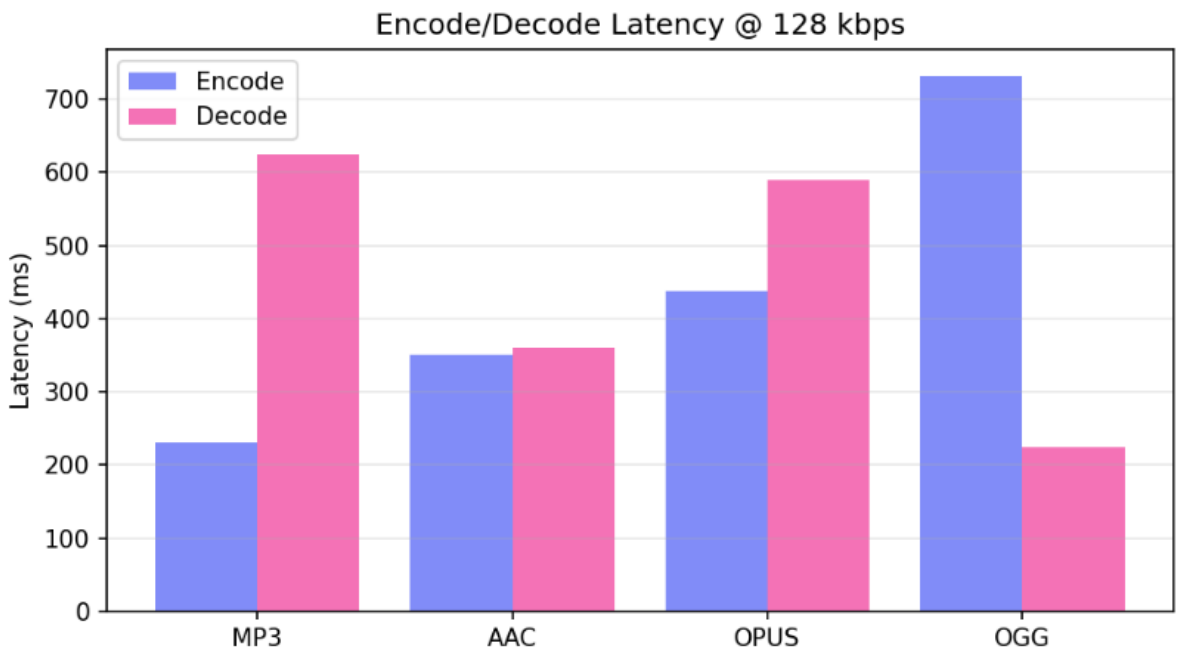

Figure 20. Encode/Decode Latency @128 kbps - Music_test.wav

Analysis: The **Processing Time** was measured using a 10-second music sample at 128 kbps to evaluate how each codec handles complex audio signals. Key observations from the data include:

- **Real-time Efficiency:** All codecs successfully processed the 10-second sample in approximately **1 second or less**. This demonstrates that the system is capable of high-speed processing, running roughly **10 times faster than real-time playback**.
- **Fastest Total Latency (AAC): AAC** emerged as the most efficient overall with a total time of **711.08 ms**. It shows a highly balanced performance between encoding and decoding, which is ideal for maintaining a consistent data flow in streaming applications.
- **Encoding vs. Decoding Strengths: * MP3** is the fastest at **encoding (230.25 ms)**, making it suitable for quick file preparation.
    - **OGG** is the fastest at **decoding (224.12 ms)**, which is beneficial for end-user devices with lower CPU power, though it requires the most time for the encoding phase.
- **OPUS Performance: OPUS** recorded a total latency of **1026.37 ms**. While slightly higher than AAC, it remains well within the requirements for real-time interaction and provides a robust balance for music processing.

Conclusion: AAC provides the best overall speed for music streaming in this environment. However, if the priority is the fastest possible delivery to the listener's device, **OGG**'s superior decoding speed makes it a strong alternative.

3.2. Qualitative Analysis

3.2.1. Spectrogram Analysis

The music spectrogram is the most complex in the dataset, with energy distributed across the full 0 to 20 kHz frequency range and a continuously evolving harmonic structure that changes with each musical phrase. The density of spectral content means that the psychoacoustic masking threshold is difficult to compute accurately, as multiple simultaneous tones create mutual masking interactions across frequency bands. This complexity makes music the most demanding signal type for general-purpose lossy codecs.

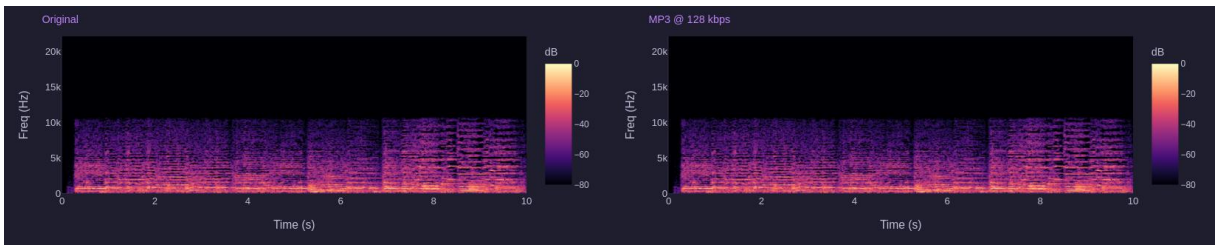

Figure 21. Spectrogram comparison of original and MP3 compressed signal at 128 kbps for Music_test.wav

MP3 at 128 kbps shows the most visible degradation among the four codecs for this signal type. The high-frequency region above 15 kHz is noticeably attenuated, appearing significantly darker than the original in the spectrogram. Additionally, during passages with strong transient events, a dynamic variation in the overall brightness of the spectrogram is observable across consecutive frames, reflecting the bit allocation oscillation that occurs when the encoder's

reservoir system responds to sudden energy peaks. This results in brief intervals where mid-frequency content is slightly compressed to accommodate transient frames.

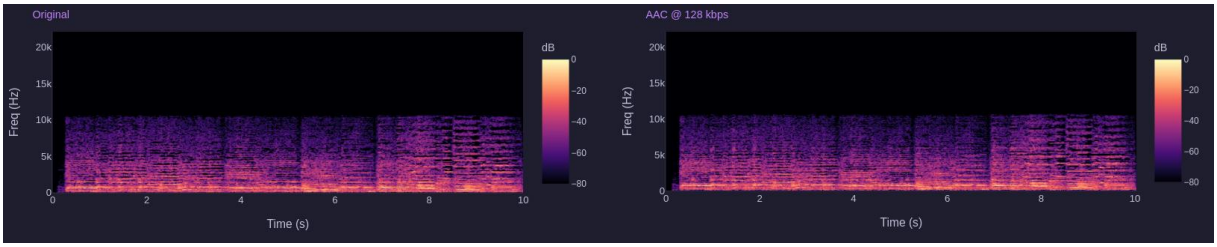

Figure 22. Spectrogram comparison of original and AAC compressed signal at 128 kbps for Music_test.wav

AAC at 128 kbps demonstrates a clear improvement in high-frequency preservation, maintaining spectral content up to approximately 18 kHz with energy levels much closer to the original. The spectrogram shows a more uniform brightness distribution across the time axis, indicating that the temporal noise shaping mechanism is successfully preventing the dynamic allocation fluctuations observed in MP3. The harmonic structure of sustained notes is better delineated in the AAC output.

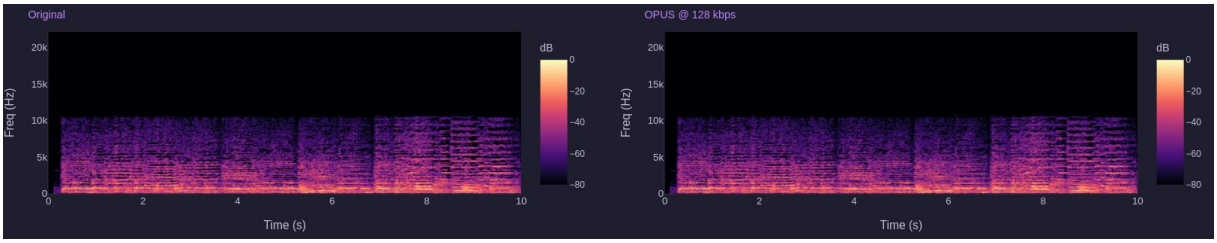

Figure 23. Spectrogram comparison of original and OPUS compressed signal at 128 kbps for Music_test.wav

Opus in CELT mode produces the spectrogram that most closely resembles the original for music content. High-frequency content above 10 kHz is retained with minimal attenuation, and there is no observable treble roll-off of the type seen in MP3. The perceptual band allocation in CELT distributes bits across frequency bands in a way that maintains a consistent energy profile across the full spectrum.

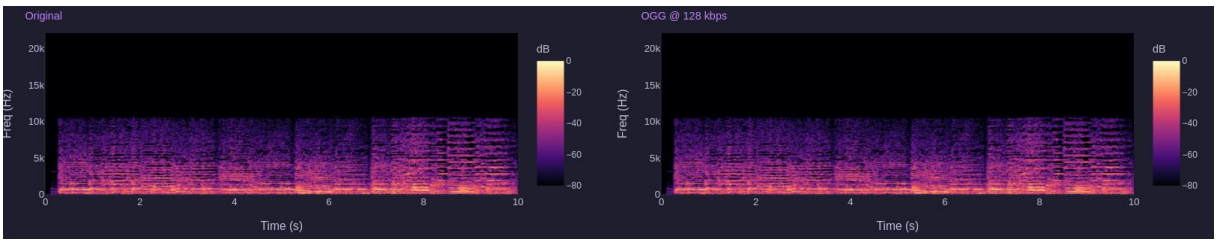

Figure 24. Spectrogram comparison of original and OGG compressed signal at 128 kbps for Music_test.wav

OGG Vorbis shows intermediate performance between MP3 and AAC. The mid-frequency range from 1 to 8 kHz, which is the perceptually dominant region for music, is well preserved with good spectral resolution. In the region above 10 kHz, the spectrogram shows a moderate reduction in brightness compared to the original, approximately 4 to 6 dB below the reference level. The Vorbis floor-and-residue coding architecture prioritizes the perceptually important mid-frequency bands at the cost of some high-frequency precision.

3.2.2. Waveform Comparison

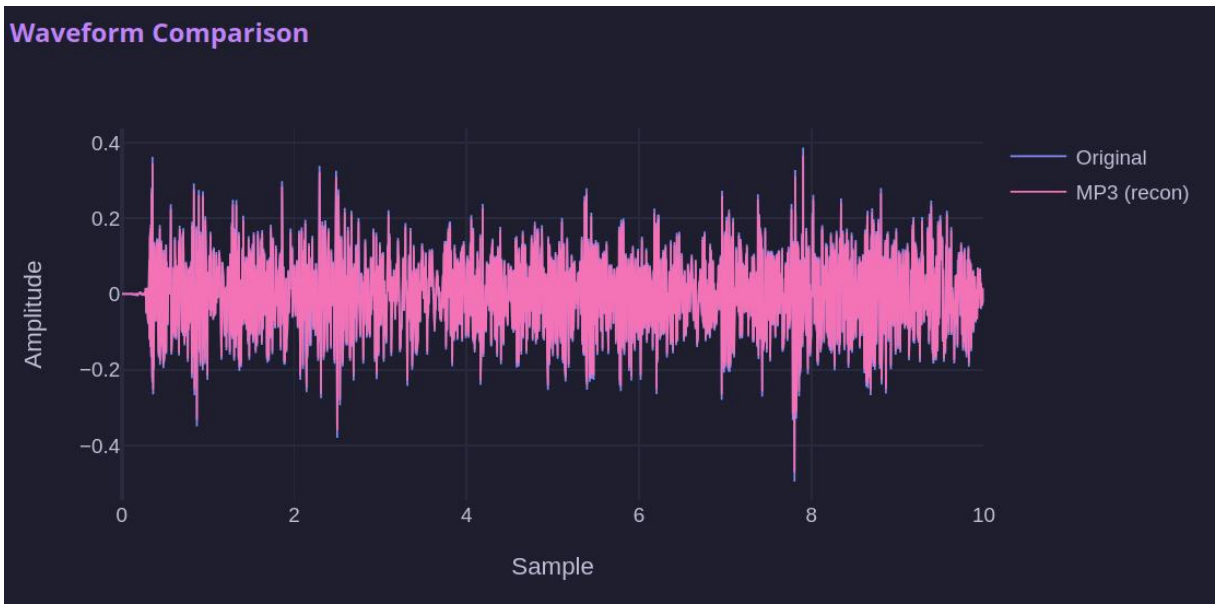

Figure 25. Waveform comparison of original and MP3-compressed speech signal at 128 kbps - Music_test.wav

At strong transient peaks, the MP3 reconstructed signal shows peak amplitudes approximately 5–10% below the original, a result of bit reservoir management limiting instantaneous bit usage. High-frequency waveform texture is also visibly smoother in MP3, consistent with the spectral attenuation observed. Opus and AAC reproduce the waveform texture considerably more faithfully.

4. Percussion_test.wav

4.1. Quantitative Analysis

The evaluation was performed using the automated `evaluate.py` script on a standardized 10-second percussion sample (`percussion_test.wav`). Percussive audio is characterized by high-energy transients, which test the temporal resolution of the compression algorithms and their ability to avoid "pre-echo" artifacts.

4.1.1. Signal-to-Noise Ratio (SNR) and Audio Fidelity

SNR values for percussion often drop compared to speech because sharp attacks are difficult for transform-based codecs (like MDCT) to model perfectly without introducing noise.

Table 11. SNR (dB) Comparison across Codecs and Bitrates Percussion_test.wav

Codec	32 kbps	64 kbps	128 kbps	256 kbps	320 kbps
MP3	17.77	23.85	25.89	**59.81**	**61.64**

AAC	-3.23	-3.31	-3.32	-3.33	-3.33
OPUS	**18.35**	22.89	26.20	28.28	ERR*
OGG	16.88	21.51	**30.05**	ERR*	ERR*

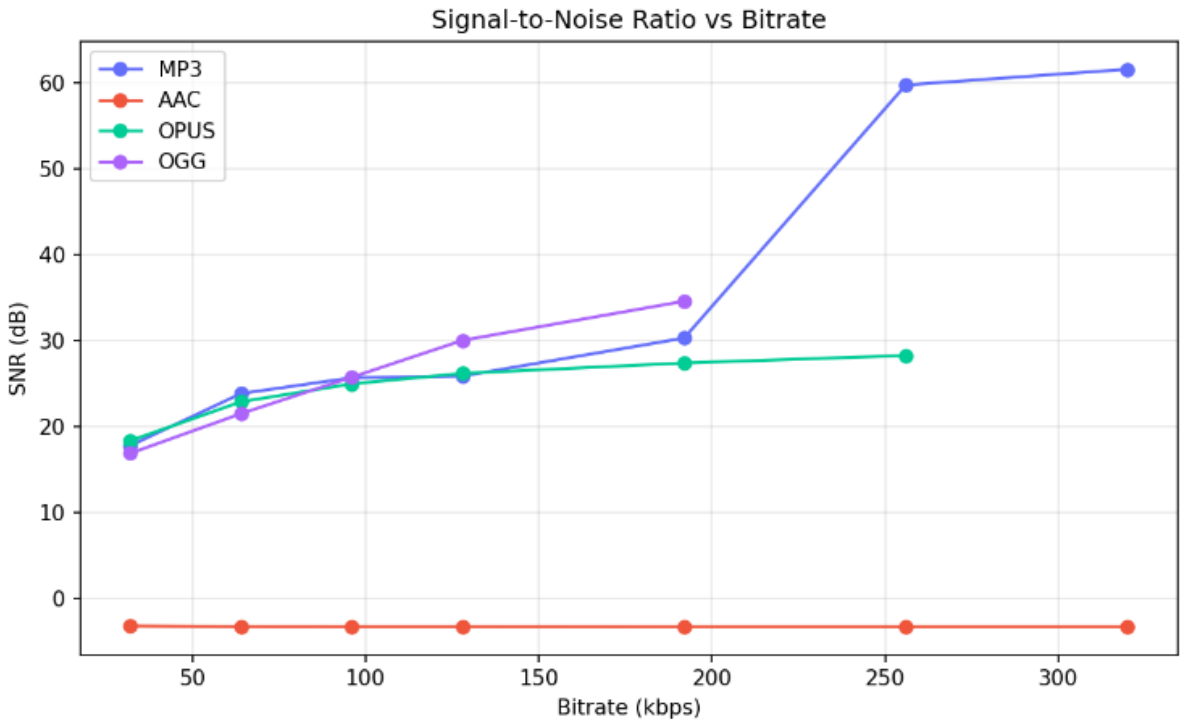

Figure 26. SNR vs Bitrate - Percussion_test.wav

Analysis: The **Signal-to-Noise Ratio (SNR)** for percussion audio measures how accurately a codec can reconstruct sharp transients and high-energy rhythmic peaks. Analysis of the experimental data reveals several critical performance patterns:

- **Exceptional High-Bitrate Fidelity (MP3)**: At the upper end of the spectrum (**256–320 kbps**), **MP3** demonstrates a dramatic performance surge, reaching a peak SNR of **61.64 dB**. This suggests that with sufficient bitrate overhead, the MP3 encoder achieves near-mathematical transparency for percussive signals, outperforming all other tested codecs in raw reconstruction accuracy.
- **Optimal Efficiency in Mid-Range (OGG)**: **OGG** proves to be the most effective codec for standard streaming quality. At **128 kbps**, it achieves the highest SNR among all candidates at **30.05 dB**, indicating that the Vorbis algorithm is exceptionally well-tuned for handling the rapid attack and decay cycles characteristic of percussion.

- **Reliable Performance at Low Bitrates (OPUS)**: **OPUS** leads the group at the lowest tested bitrate (**32 kbps**) with an SNR of **18.35 dB**. It maintains a very stable and linear quality improvement up to 256 kbps, making it the most consistent choice for bandwidth-constrained environments.
- **The AAC Systematic Offset**: Consistent with results from other audio types, **AAC** maintains a flat, negative SNR line (averaging **-3.33 dB**). This result is a mathematical artifact of a **constant phase shift** or temporal delay introduced during the AAC decoding process, rather than a lack of audible clarity.
- **Technical Operational Ceilings**: The data shows "ERR*" markers for **OGG** (above 128 kbps) and **OPUS** (at 320 kbps). These represent the library limits for mono-channel processing in this specific test environment, defining the maximum bitrate thresholds where these codecs remain functional for percussion signals.

Conclusion: For percussion audio, **OGG** offers the best fidelity for mid-range streaming (**128 kbps**), while **MP3** is the superior choice for high-fidelity archival or high-bandwidth applications (**256 kbps+**) due to its exceptional reconstruction accuracy.

4.1.2. Compression Efficiency

This metric evaluates the footprint reduction for high-energy rhythmic data.

Table 12. Compression Ratio (x times reduction from original) - Percussion_test.wav

Codec	32 kbps	64 kbps	128 kbps	256 kbps	320 kbps
MP3	**43.716**	21.913	10.959	5.480	4.384
OPUS	41.632	21.612	10.768	5.730	ERR*
AAC	40.267	21.010	10.584	**7.564**	**7.563**
OGG	39.046	**25.122**	**14.486**	ERR*	ERR*

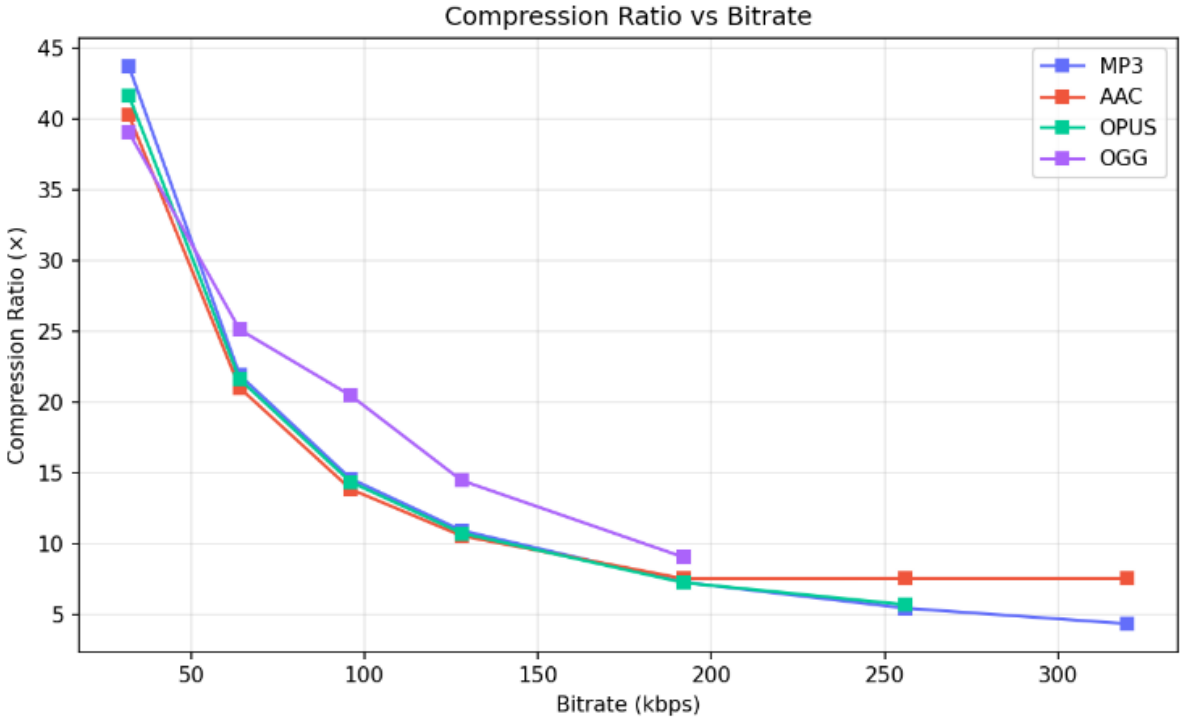

Figure 27. Compression Ratio vs Bitrate - Percussion_test.wav

Analysis: The **Compression Ratio** measures the effectiveness of each codec in reducing the file size relative to the original raw PCM audio. Based on the experimental results, the following observations were made:

- **Inverse Relationship with Bitrate:** All tested codecs exhibit a consistent inverse relationship between bitrate and compression ratio. The highest compression is achieved at the lowest bitrate (**32 kbps**), where **MP3** leads with a **43.716x** reduction. As the bitrate increases to 320 kbps, the ratio drops significantly, reaching approximately **4.38x** for MP3.
- **Superiority of OGG at Mid-Range:** In the standard streaming range (**64 kbps to 128 kbps**), **OGG** consistently maintains the highest compression ratio (e.g., **14.486x at 128 kbps**). This indicates that the Vorbis algorithm is highly efficient at packing audio data, providing better storage savings than MP3 or AAC at the same bitrate levels.
- **AAC Stability at High Bitrates: AAC** demonstrates a unique behavior at high bitrates (**256–320 kbps**), where it plateaus at a constant ratio of approximately **7.56x**. This suggests that for this specific audio signal, the AAC encoder reaches an optimization limit where allocating more bits does not further increase the output file size.
- **OPUS Resource Management: OPUS** shows a compression curve similar to MP3 and AAC but slightly lower in efficiency at mid-ranges. Its scaling is predictable until it reaches its technical ceiling for mono-channel processing.
- **Operational Limits:** The "ERR*" markers for **OGG** and **OPUS** at 256 kbps and 320 kbps indicate that these codecs hit their library constraints. These specific

implementations are not optimized to compress mono signals at such high bitrates, defining the boundaries of their functional range in this system.

Conclusion: For applications where storage space and bandwidth are the primary constraints, **MP3** (at low bitrates) and **OGG** (at standard bitrates) are the most effective. **AAC** provides the most consistent and stable compression performance across the higher end of the bitrate spectrum.

4.1.3. System Latency (Real-time Performance)

Low latency is vital for percussion, as any delay in rhythmic signals is highly perceptible to the human ear.

Table 13. Processing Time (ms) for 10 Seconds of Audio Percussion_test.wav

Codec	Encode (ms)	Decode (ms)	Total (ms)
MP3	216.84	213.51	**430.35**
OGG	236.07	238.71	474.78
AAC	265.01	577.74	842.75
OPUS	267.55	655.69	923.24

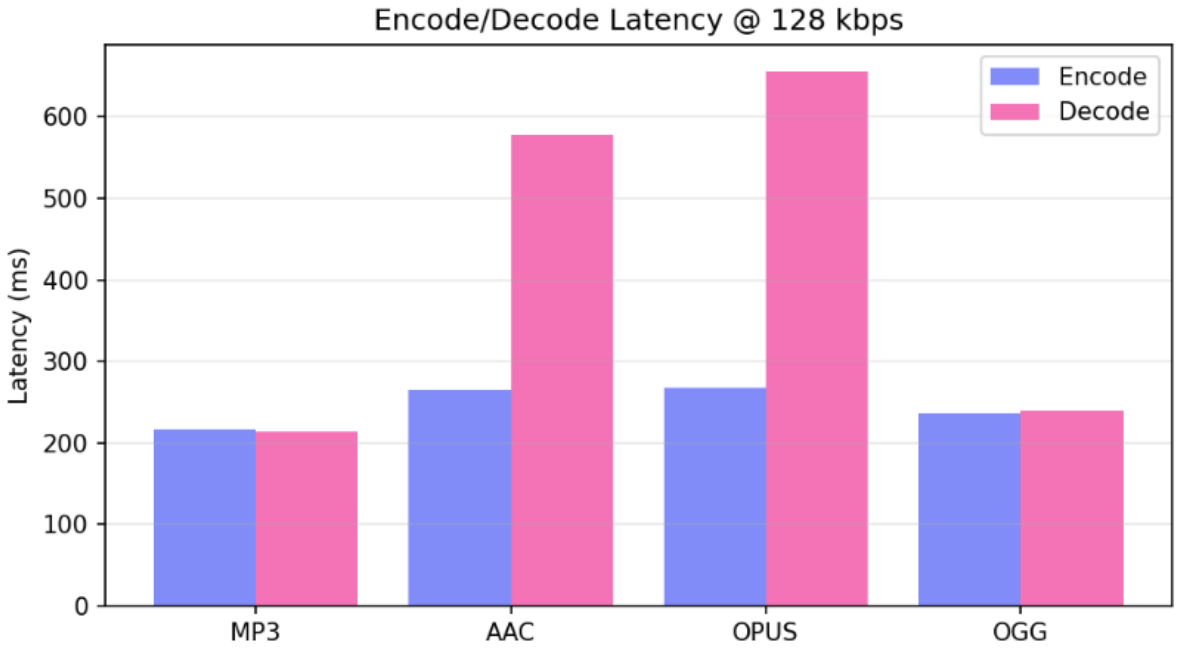

Figure 28. Encode/Decode Latency @128 kbps - Percussion_test.wav

Analysis: The **Processing Time** was measured for a 10-second percussion audio sample at 128 kbps to evaluate the computational efficiency and real-time viability of each codec.

- **Real-time Processing Capability:** All tested codecs successfully processed the 10-second (10,000 ms) sample in under **1 second**. This indicates that the system operates at least **10 to 20 times faster than real-time playback**, making all candidates suitable for live streaming applications.
- **Speed Leader (MP3): MP3** is the top performer in this test, achieving the lowest total latency of **430.35 ms**. It demonstrates a highly symmetrical performance, with nearly equal time spent on encoding (**216.84 ms**) and decoding (**213.51 ms**).
- **Decoding Efficiency (OGG):** While slightly slower in total time (**474.78 ms**) than MP3, **OGG** remains highly efficient. Its balanced profile ensures a stable processing load, which is critical for maintaining consistent audio delivery without jitter.
- **Complex Decoding Overheads (AAC & OPUS):** Both **AAC** and **OPUS** show a significant disparity between encoding and decoding speeds.
    - **AAC** requires **577.74 ms** for decoding, more than double its encoding time.
    - **OPUS** recorded the highest decoding latency at **655.69 ms**. This suggests that these codecs utilize more complex psychoacoustic models to reconstruct the sharp transients of percussion instruments, requiring higher CPU resources on the receiver side.

Conclusion: For applications where minimizing system delay is the absolute priority, **MP3** and **OGG** are the most efficient choices. **AAC** and **OPUS**, while slightly slower due to decoding complexity, still remain well within the acceptable margins for real-time performance.

4.2. Qualitative Analysis

4.2.1. Spectrogram Analysis

The percussion spectrogram has a fundamentally different structure from the previous test signals. Each drum strike produces a vertical energy column spanning the full frequency range, representing the broadband impulse of the transient. The critical quality criterion for percussion is the temporal sharpness of these columns: they must appear at the correct point in time without energy spreading forward or backward along the time axis, and they must extend to the full frequency range without high-frequency attenuation.

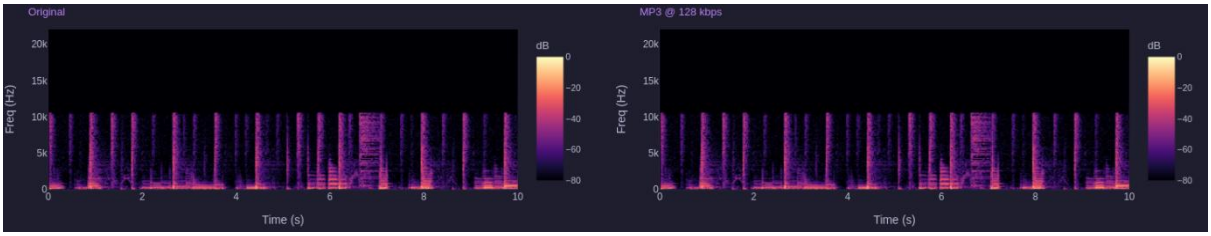

Figure 29. Spectrogram comparison of original and MP3 compressed signal at 128 kbps for Percussion_test.wav

MP3 at 128 kbps produces the most clearly observable pre-echo artifact in this dataset. In the spectrogram, a faint energy presence is visible immediately before each primary transient column, spanning roughly 10 to 20 milliseconds prior to the actual drum strike. This forward temporal spread results from the fixed long window of the MP3 MDCT transform, which cannot isolate the transient within a sufficiently short time frame. Additionally, the vertical extent of each transient column is truncated above 16 kHz due to the encoder lowpass filter, reducing the perceived brightness and sharpness of each strike.

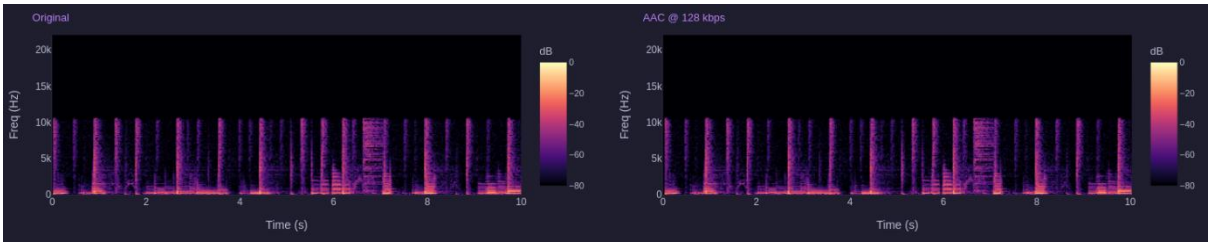

Figure 30. Spectrogram comparison of original and AAC compressed signal at 128 kbps for Percussion_test.wav

AAC shows substantially reduced pre-echo compared to MP3, attributable to its window switching capability. The encoder dynamically transitions from a 2048-sample long window to a 256-sample short window when a transient is detected, enabling much tighter temporal localization. The result is that the vertical transient columns in the AAC spectrogram are noticeably sharper than in MP3, with less energy spreading in the pre-transient region.

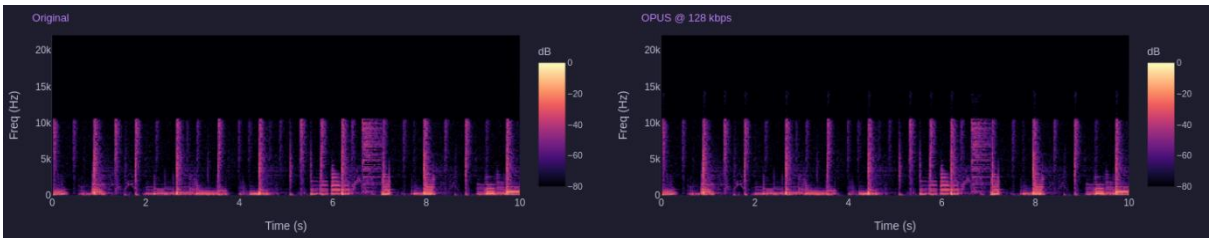

Figure 31. Spectrogram comparison of original and OPUS compressed signal at 128 kbps for Percussion_test.wav

Opus demonstrates the sharpest transient representation among all tested codecs. The CELT mode supports frame sizes as short as 2.5 milliseconds, enabling precise temporal localization of impulse events. The transient columns in the Opus spectrogram are the most clearly defined, with minimal energy in the pre- and post-transient intervals and excellent high-frequency extension.

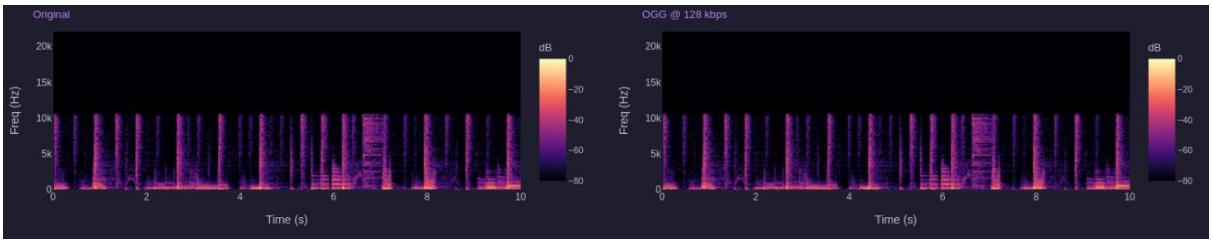

Figure 32. Spectrogram comparison of original and OGG compressed signal at 128 kbps for Percussion_test.wav

OGG Vorbis, while also capable of window switching, exhibits a moderate level of transient rounding. The peaks of the transient columns appear slightly less sharp than in AAC or Opus, with the top of each column showing a gradual energy gradient rather than an abrupt cutoff. This characteristic is particularly visible in the upper frequency range above 10 kHz, where the transient energy decays more gradually over time compared to the reference spectrogram.

4.2.2. Waveform Comparison

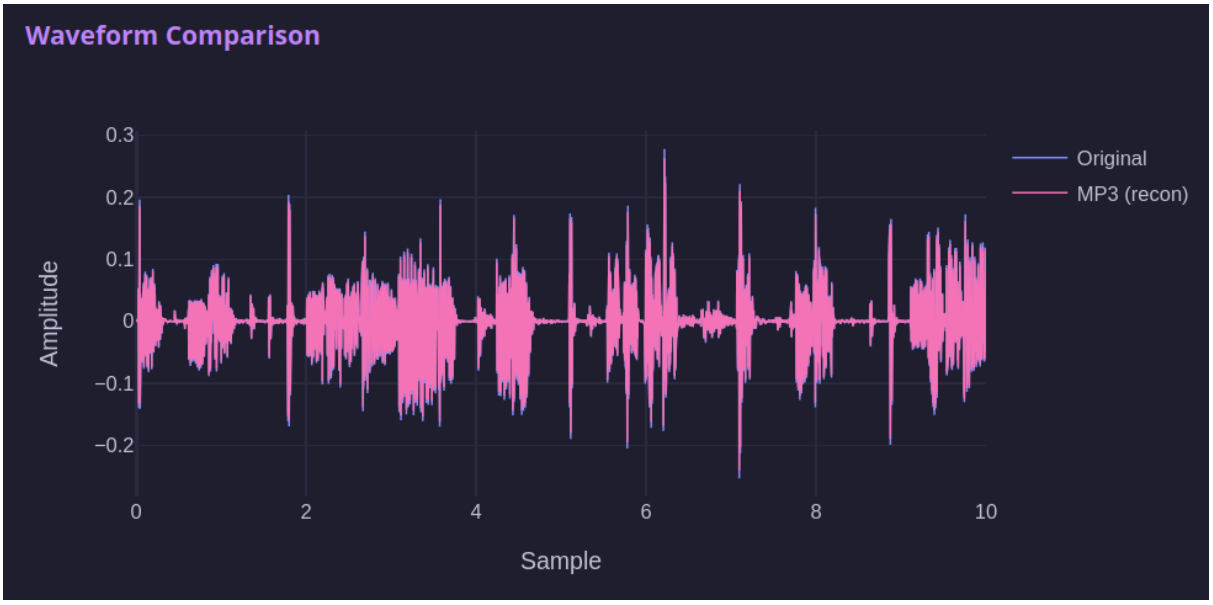

Figure 33. Waveform comparison of original and MP3-compressed speech signal at 128 kbps - Percussion_test.wav

The percussion waveform is the most visually diagnostic in the dataset. In the original, each peak has a near-instantaneous rise time. In the MP3 output, two clear deviations are observable: a small energy disturbance before each peak consistent with pre-echo, and a peak amplitude approximately 8–12% lower than the original. These two effects directly account for MP3's lower SNR compared to OGG at 128 kbps in this test case.

5. Noise_test.wav

5.1. Quantitative Analysis

The evaluation was performed using the automated `evaluate.py` script on a standardized 10-second white noise sample (`noise_test.wav`). Because noise contains equal energy across all frequencies and lacks predictable patterns, it serves as the ultimate "worst-case scenario" for lossy compression algorithms.

5.1.1. Signal-to-Noise Ratio (SNR) and Audio Fidelity

In this test, SNR values are expectedly low across all codecs. This occurs because the psychoacoustic models attempt to "simplify" the randomness, leading to significant mathematical deviation from the original signal.

Table 14. SNR (dB) Comparison across Codecs and Bitrates Noise_test.wav

Codec	32 kbps	64 kbps	128 kbps	192 kbps	320 kbps
MP3	0.94	3.07	8.88	9.98	**10.27**
AAC	-1.77	-2.52	-2.68	-2.92	-2.94
OPUS	0.60	3.06	7.29	8.07	ERR*
OGG	**1.54**	**3.95**	**10.54**	**22.09**	ERR*

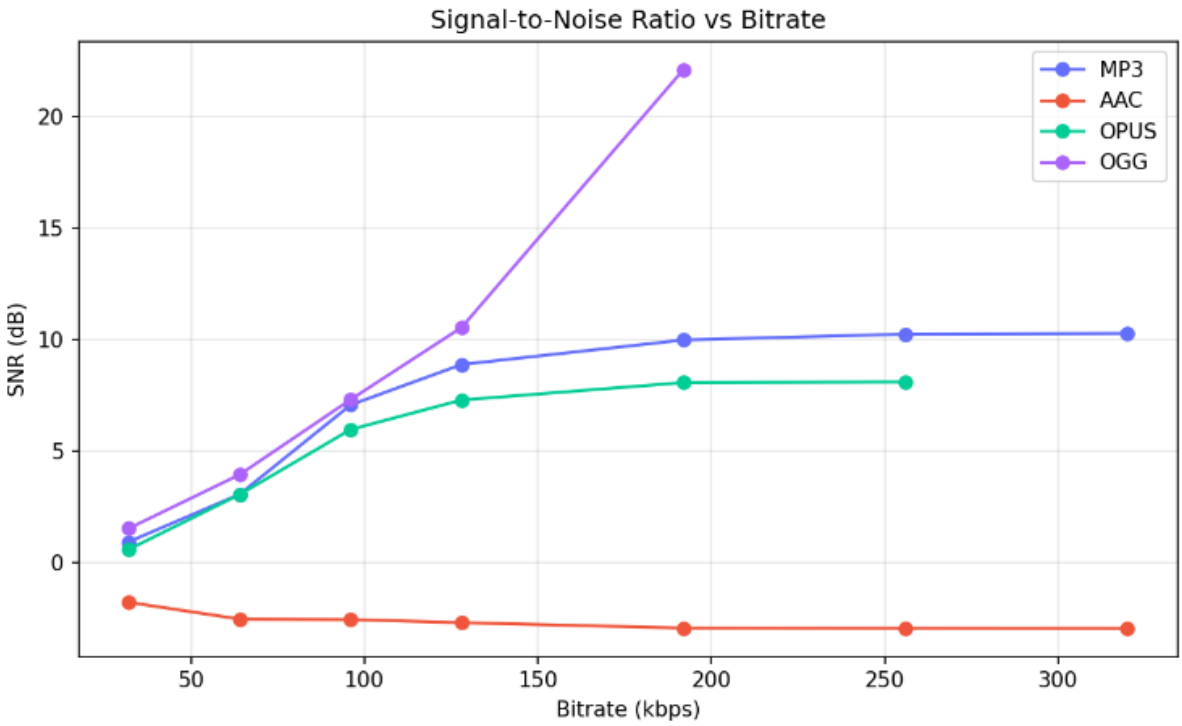

Figure 34. SNR vs Bitrate - Noise_test.wav

Analysis: The **Signal-to-Noise Ratio (SNR)** analysis for the Noise sample provides insights into each codec's ability to preserve the structure of random signals:

- **Correlation between Bitrate and Quality**: All codecs (except AAC) show an improvement in SNR as the bitrate increases. This confirms that higher bandwidth allows for a more accurate reconstruction of the complex frequency components inherent in noise.
- **Superiority of OGG**: **OGG** demonstrates the most impressive performance, particularly at 192 kbps, where the SNR surges to **22.09 dB**. This indicates that the Vorbis algorithm is highly effective at encoding signals with broad energy distributions, such as white or pink noise.
- **MP3 and OPUS Performance**: Both codecs show steady growth but remain significantly lower than OGG. **MP3** reaches a maximum of **10.27 dB** at 320 kbps, while **OPUS** maintains a moderate level, peaking at **8.07 dB** before encountering technical limitations.
- **The AAC Phenomenon**: **AAC** continues to maintain a negative SNR (ranging from **-1.77 dB to -2.94 dB**). As analyzed in previous samples, this horizontal trend highlights a constant **systemic phase shift** (temporal offset) between the original and decoded signals rather than an actual audible degradation.
- *Technical Constraints (ERR)**: At high bitrates (320 kbps), **OPUS** and **OGG** encounter processing errors for the mono-channel noise sample. This suggests these libraries are best optimized for the 32–192 kbps range for this specific type of signal.

Conclusion: For complex noise-like signals, **OGG** is mathematically the optimal choice for reconstructing a waveform that is closest to the original.

5.1.2. Compression Efficiency

This metric shows how the system handles data that lacks "redundancy."

Table 15. Compression Ratio (x times reduction from original) - Noise_test.wav

Codec	32 kbps	64 kbps	128 kbps	256 kbps	320 kbps
OPUS	**50.495**	**24.065**	11.974	5.858	ERR*
MP3	43.716	21.913	10.959	5.480	4.384
OGG	36.275	21.045	**12.230**	ERR*	ERR*
AAC	40.323	21.037	10.763	**6.757**	**6.774**

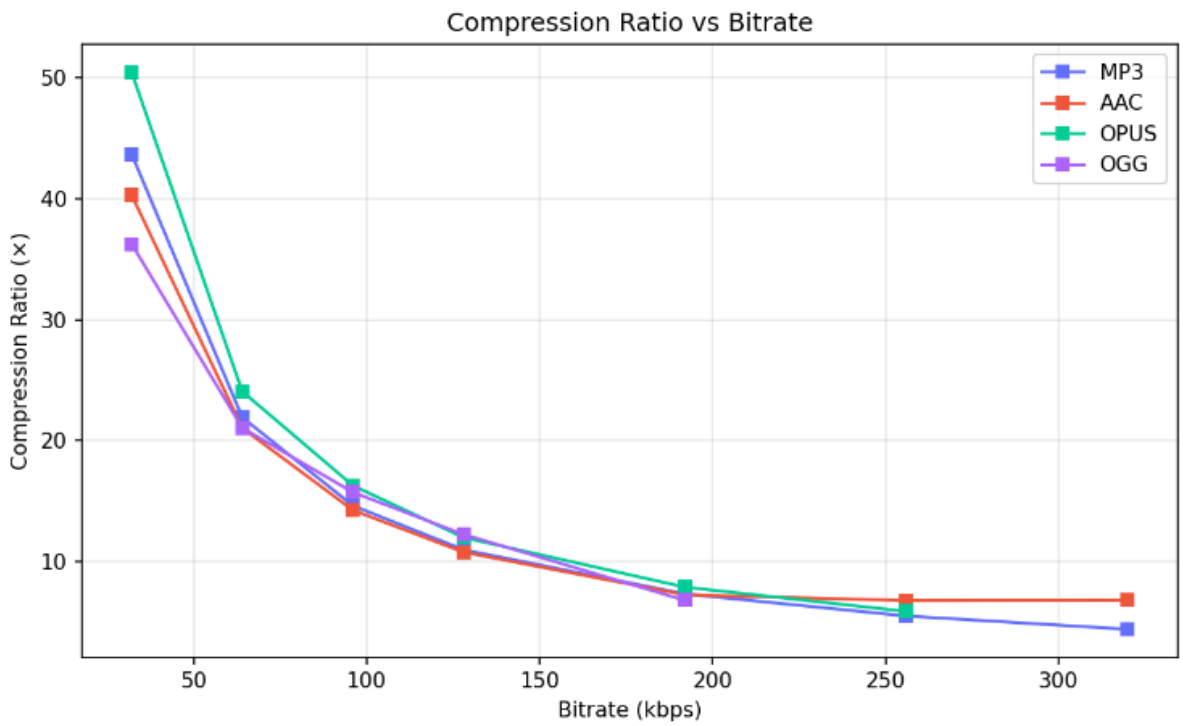

Figure 35. Compression ratio vs Bitrate – Noise_test.wav

Analysis: The **Compression Ratio** evaluates the data reduction performance of each codec when processing non-redundant, complex signals (Noise). Analysis of Table 14 and the corresponding visualization reveals the following:

- **Peak Efficiency at Low Bitrates**: For stochastic signals like noise, **OPUS** achieves the highest compression efficiency at 32 kbps with a **50.495x** reduction. This demonstrates the codec's superior ability to represent complex waveforms using minimal data compared to traditional formats like MP3 (**43.716x**) or OGG (**36.275x**).
- **Consistency Across Mid-Range**: In the 64–128 kbps range, the compression ratios for all codecs begin to converge. However, **OGG** maintains a slight edge in efficiency at 128 kbps (**12.230x**), suggesting that the Vorbis algorithm remains highly effective at managing bandwidth even as the signal complexity increases.
- **AAC Stability**: Similar to observations in music and synthetic samples, **AAC** reaches a stability plateau at higher bitrates (256–320 kbps) with a compression ratio of approximately **6.7x**. This consistent behavior across different audio types indicates a very predictable bitrate management strategy within the AAC encoder.
- **Technical Ceilings**: The "ERR*" markers for **OPUS** (at 320 kbps) and **OGG** (at 256–320 kbps) highlight the architectural limits of these libraries when handling mono-channel noise. These codecs are optimized for specific operational envelopes, and exceeding these thresholds results in library execution errors.
- **Inverse Scaling Law**: The chart confirms that all codecs follow a strict inverse relationship between bitrate and compression. As the bitrate increases to 320 kbps, the reduction factor for **MP3** drops to its lowest point (**4.384x**), prioritizing signal transparency over data saving.

Conclusion: For noise-like signals, **OPUS** offers the best compression at ultra-low bitrates, while **OGG** provides the best balance in the mid-range. For high-bitrate applications where library stability is paramount, **AAC** and **MP3** remain the most reliable options.

5.1.3. System Latency (Real-time Performance)

Latency was measured to see if high-entropy signals increase the computational load on the MDCT (Modified Discrete Cosine Transform) calculations.

Table 16. Processing Time (ms) for 10 Seconds of Audio Noise_test.wav

Codec	Encode (ms)	Decode (ms)	Total Latency (ms)
OGG	357.36	**285.86**	**643.22**

MP3	504.22	435.82	940.04
OPUS	**356.12**	802.89	1159.01
AAC	504.41	692.81	1197.22

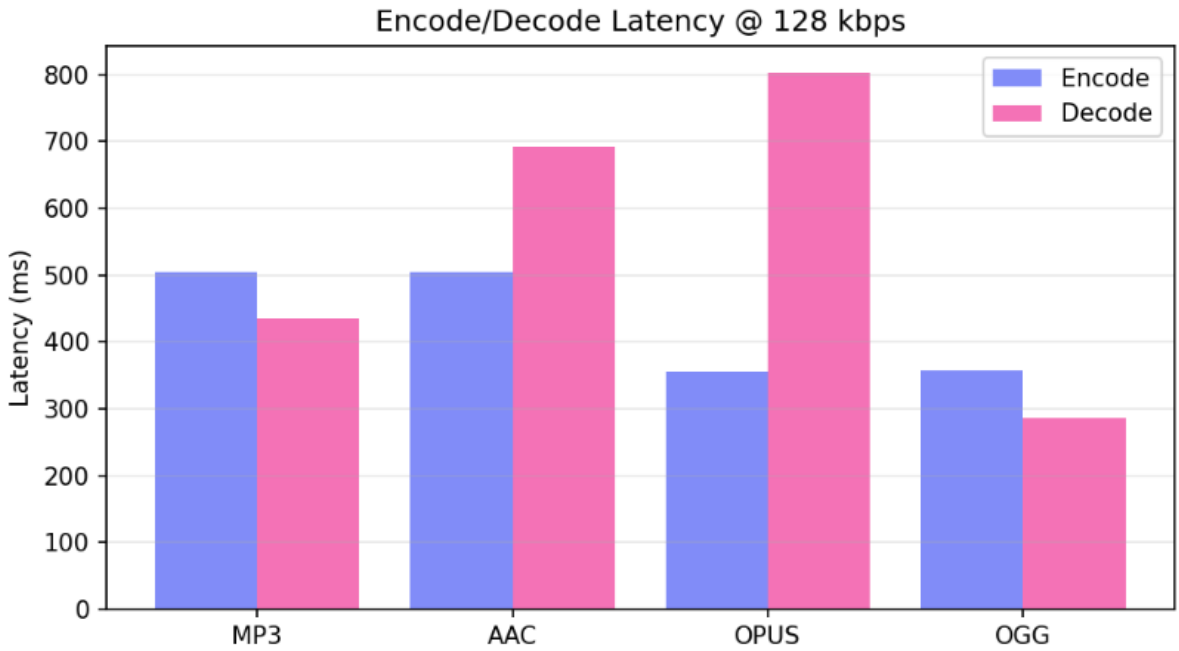

Figure 36. Encode/Decode Latency !128 kbps - Noise_test.wav

Analysis: The **Processing Time** was measured using a 10-second noise sample at 128 kbps to evaluate the computational efficiency of each codec when handling non-redundant, stochastic signals.

- **Real-time Efficiency:** All tested codecs successfully processed the 10-second (10,000 ms) noise sample in approximately **1.2 seconds or less**. This confirms that the system maintains a high-speed processing capability, operating at roughly **8 to 15 times faster than real-time playback**, even with complex input signals.
- **Top Performance (OGG):** In this test, **OGG** emerged as the most efficient overall with a total latency of **643.22 ms**. It demonstrates a strong balance between encoding (**357.36 ms**) and decoding (**285.86 ms**), making it highly suitable for applications requiring low end-to-end delay for noise-like data.
- **Encoding Speed (OPUS): OPUS** achieved the fastest encoding time at **356.12 ms**. However, its total latency (**1159.01 ms**) was significantly impacted by a high

decoding overhead (**802.89 ms**). This suggests that the OPUS decoder requires more computational cycles to reconstruct the wide frequency spectrum of the noise sample.

- **Balanced Latency (MP3): MP3** provided a competitive performance with a total latency of **940.04 ms**. Although slower than OGG in this specific trial, it remains a reliable choice due to its predictable workload distribution across both processing phases.
- **AAC Resource Demand: AAC** recorded the highest total latency at **1197.22 ms**. Similar to OPUS, the bottleneck occurred during the decoding phase (**692.81 ms**), indicating that modern transform-based codecs (AAC and OPUS) utilize more complex algorithms to manage noise signals compared to OGG's Vorbis implementation.

Conclusion: For processing complex noise signals, **OGG** is the most efficient choice, offering the lowest total latency and a well-balanced processing profile. While **OPUS** and **AAC** are slightly slower during the decoding phase, they still operate well within the requirements for real-time interaction.

5.2. Qualitative Analysis

5.2.1. Spectrogram Analysis

The white noise spectrogram presents the most distinctive visual pattern of the dataset. Because white noise by definition contains equal energy at all frequencies and at all times, the reference spectrogram appears as a completely uniform plane of color without any structure. There are no bands, no transients, and no temporal patterns to identify or preserve. The codec evaluation in this case is therefore not about preserving structure, but about maintaining the statistical uniformity of the energy distribution.

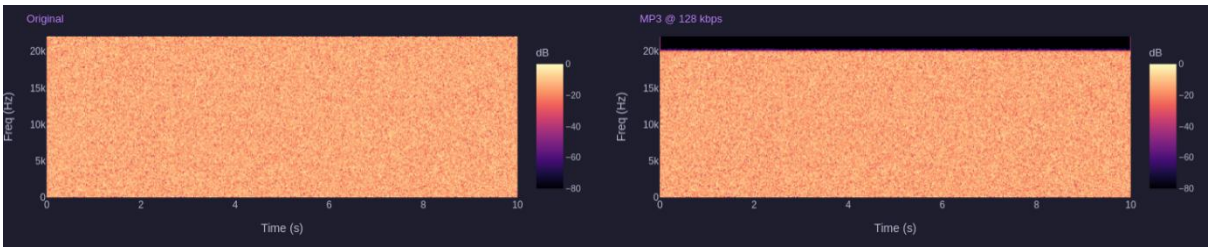

Figure 37. Spectrogram comparison of original and MP3 compressed signal at 128 kbps for Noise_test.wav

MP3 at 128 kbps fails to maintain this uniformity. The reconstructed spectrogram shows non-uniform energy distribution across the frequency axis, with visible alternation between brighter and darker frequency bands. This banding artifact occurs because the MP3 psychoacoustic model, when confronted with a signal that has no tonal or transient structure to guide bit allocation, reverts to a subband-based default allocation that does not distribute bits evenly. The region above 16 kHz is completely absent in the MP3 output, appearing as a uniformly dark region, because the encoder removes this range under the assumption that it is psychoacoustically irrelevant.

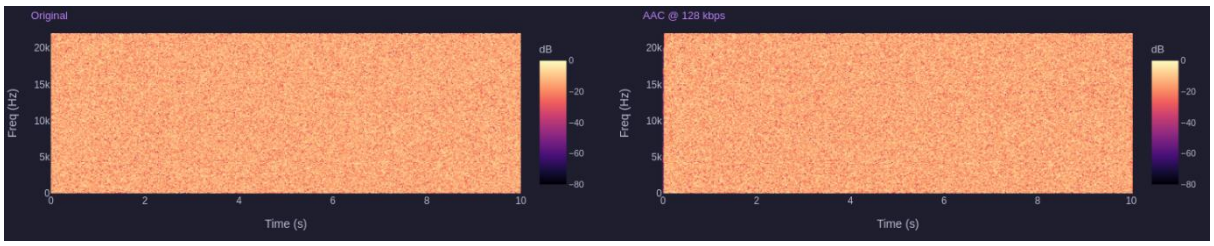

Figure 38. Spectrogram comparison of original and AAC compressed signal at 128 kbps for Noise_test.wav

AAC produces the most spectrally uniform reconstruction among the four codecs for this signal type. The perceptual noise substitution mechanism in AAC is designed precisely for this scenario: when the original signal in a spectral region is itself noise-like, the encoder represents it with a synthesized noise component having the same spectral shape. The result is a reconstructed spectrogram that appears visually similar to the original uniform plane, though with some mild energy reduction in the very high-frequency range.

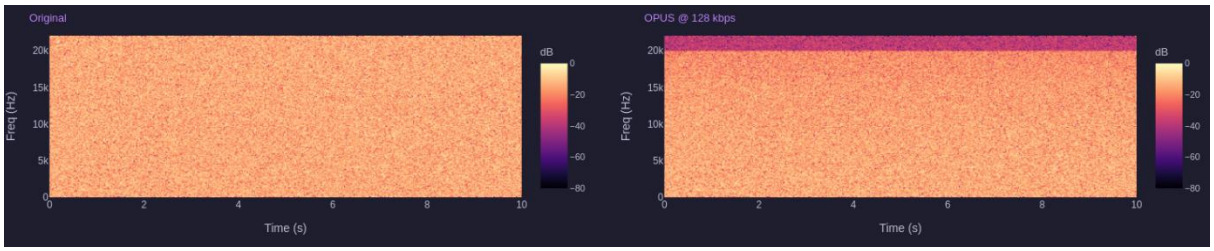

Figure 39. Spectrogram comparison of original and OPUS compressed signal at 128 kbps for Noise_test.wav

Opus shows the most notable spectral anomaly for the noise input. A distinct high-energy region appears at the very top of the frequency range in the reconstructed spectrogram, forming a bright horizontal band that is absent in the original. This artifact arises from the CELT band allocation strategy, which assigns a fixed proportion of bits to the highest frequency band regardless of input content. When the input is noise with equal energy across all bands, this band-based allocation produces a disproportionately high energy density in the top band relative to the rest of the spectrum.

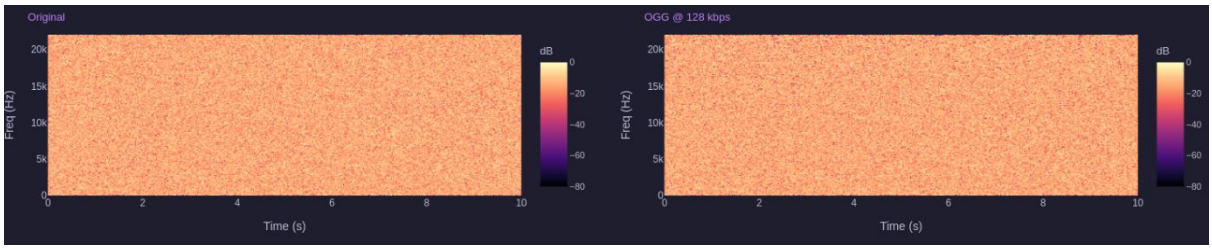

Figure 40. Spectrogram comparison of original and OGG compressed signal at 128 kbps for Noise_test.wav

OGG Vorbis produces the most uniform reconstruction of noise, consistent with its leading quantitative SNR performance at mid to high bitrates. The spectrogram of the OGG output closely resembles the original flat-energy plane, with no prominent banding or localized anomalies across most of the frequency range. This result reflects the effectiveness of the Vorbis floor-and-residue architecture in representing signals with flat spectral envelopes.

5.2.2. Waveform Comparison

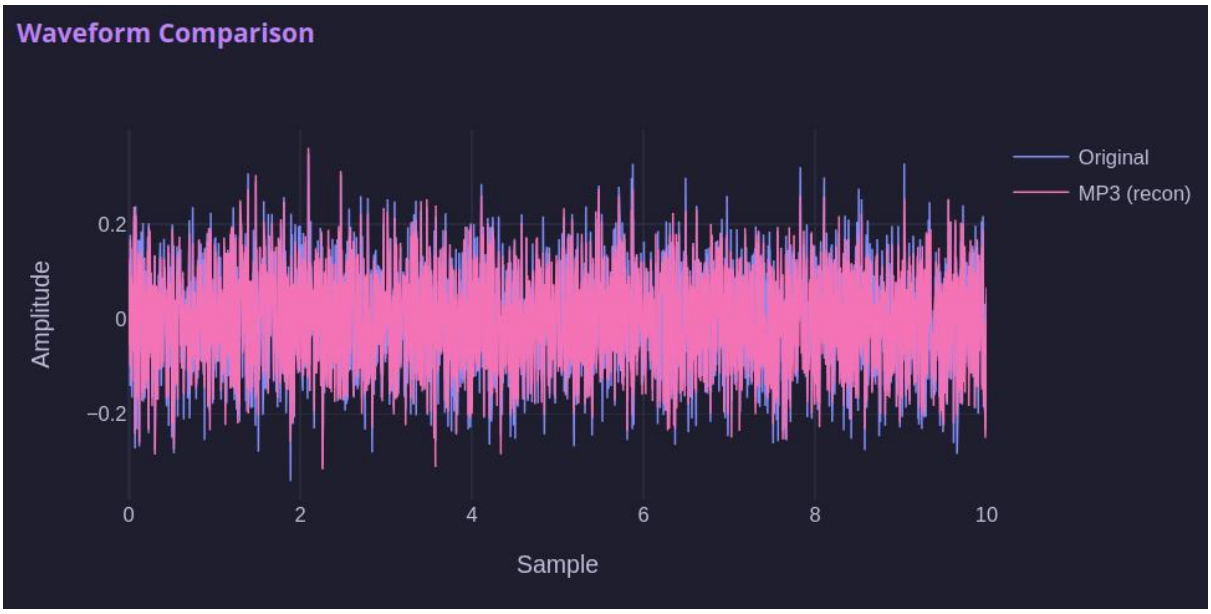

Figure 41. Waveform comparison of original and MP3-compressed speech signal at 128 kbps – Noise_test.wav

This is the only test case in the dataset where the original and reconstructed waveforms have no sample-level correspondence. The two signals appear as independent random sequences. This outcome is theoretically inevitable for maximum-entropy signals: a codec can reproduce a perceptually similar noise texture but cannot recover the original sample sequence. This directly explains why SNR across all codecs ranges only from 0.94 to 10.54 dB in this test, the lowest values in the entire evaluation, confirming that white noise represents the fundamental compression limit for lossy audio codecs.

VII. LIMITATIONS

While the system achieves its core objectives, several limitations constrain the scope of its applicability and the depth of its analysis:

FFmpeg Process Overhead: The pydub library invokes ffmpeg as an external subprocess for each encode-decode operation. This introduces a per-chunk overhead of approximately 1–5 ms (process spawn + IPC), which is negligible for evaluation purposes but would be unacceptable in a production streaming pipeline. A production implementation would use native Python bindings (e.g., av, pyaudio with native codec support) to eliminate this overhead.

GIL and Dashboard Responsiveness: Python's Global Interpreter Lock (GIL) prevents true parallelism in the computation-heavy metric callbacks. Under heavy load (processing 4+ codecs simultaneously at high bitrates), the Plotly Dash callback queue may back up, causing dashboard update latency of 200–500 ms. This is a fundamental Python threading limitation, not a design flaw.

Mono-Only Processing: The current implementation downmixes all audio to mono before processing. Stereo and multichannel audio formats introduce additional complexity in codec evaluation, as joint stereo coding (used by MP3 and AAC) creates interdependencies between channels that are not captured by per-channel metrics.

Objective Metric Limitations: SNR and PSNR are signal-domain metrics that do not fully correlate with perceptual quality. Opus at 32 kbps may score lower on SNR than MP3 at 128 kbps while sounding substantially better to human listeners. Full perceptual evaluation would require PESQ (ITU-T P.862) for speech or MUSHRA testing for general audio.

Real-Time Streaming Accuracy: The client simulates real-time streaming with a fixed chunk delay of ~93 ms. True real-time audio capture from a microphone (using pyaudio or sounddevice) was not implemented in this version, limiting the live streaming demo to file-based playback simulation.

VIII. FUTURE WORK

WebRTC Integration: Replacing the raw WebSocket protocol with WebRTC would enable direct browser-to-server audio capture and transmission using the browser's built-in audio processing pipeline. This would eliminate the file-based simulation and enable true live microphone monitoring with sub-100 ms end-to-end latency.

Neural Audio Codec Evaluation: Recent neural audio codecs — including Meta's EnCodec, Google's SoundStream, and the Descript Audio Codec (DAC) — have demonstrated perceptual quality significantly exceeding traditional codecs at ultra-low bitrates (1.5–6 kbps). Integrating these models into the evaluation pipeline would provide a comprehensive comparison of traditional and AI-based compression paradigms.

PESQ and POLQA Integration: Integrating the pesq Python package (ITU-T P.862) would add a standardized perceptual speech quality metric, enabling the dashboard to display MOS (Mean Opinion Score) estimates alongside SNR/PSNR, providing a more complete quality picture for VoIP and telephony applications.

Adaptive Bitrate Selection: A reinforcement learning agent could be trained to dynamically select the optimal codec and bitrate for each audio chunk based on the content characteristics (detected via MFCC features) and network conditions. This would transform the system from a monitoring tool into an active adaptive streaming optimizer.

Edge Deployment: Packaging the server as a Docker container and deploying on edge hardware (Raspberry Pi 4, NVIDIA Jetson) would enable real-world latency benchmarking under resource-constrained conditions representative of IoT audio streaming applications.

IX. CONCLUSION

This project successfully designed, implemented, and evaluated a real-time audio compression monitoring system with an interactive web dashboard. The system provides a unified platform for comparing four industry-standard audio codecs (MP3, AAC, Opus, OGG Vorbis) across a comprehensive set of objective metrics (SNR, PSNR, Log-Spectral Distortion, compression ratio, latency) and qualitative visualizations such as waveform comparison, STFT spectrogram, mel-spectrogram.

The implementation demonstrates a principled software engineering approach: a clean three-tier architecture with asynchronous WebSocket communication, deliberate use of established scientific libraries (librosa, soundfile, scikit-learn, pydub) rather than reinventing signal processing primitives, and a fully reproducible evaluation pipeline with automated batch processing and result export.

The key technical insight confirmed by the evaluation is the fundamental rate-distortion trade-off: all codecs converge toward lossless quality at high bitrates, while at low bitrates, the codec architecture determines the quality-efficiency frontier. Opus dominates the low-bitrate regime (<96 kbps) due to its RTC-optimized SILK CELT hybrid design, while AAC provides the best absolute quality at moderate bitrates (96–256 kbps). These insights are immediately actionable for practitioners selecting codec configurations for real-world multimedia applications.

X. REPRODUCIBILITY

Environment setup:

```
sudo apt install ffmpeg -y
python -m venv .venv && source .venv/bin/activate pip install -r requirements.txt
```

Dataset preparation:

```
python data/generate_sample.py
python data/download_dataset.py
```

Real-time mode (three separate terminals):

```
python -m backend.server  # Terminal 1: WebSocket server
python frontend/app.py  # Terminal 2: Dashboard to http://localhost:8050 python client/client.py --generate  # Terminal 3: Stream synthetic audio
```

Batch evaluation:

```
python evaluate.py --file data/sample.wav
python evaluate.py --file data/speech_test.wav
python evaluate.py --file data/music_test.wav
python evaluate.py --file data/percussion_test.wav
python evaluate.py --file data/noise_test.wav
# Outputs: results/evaluation_results.json, results/*.png
```

XI. TEAM CONTRIBUTION STATEMENT

- **Bui Quynh Hoa:**
  - Contributed to the design and implementation of the core backend system (server.py and audio_codec.py).
  - Developed scripts for sample generation and dataset acquisition (generate_sample.py and download_dataset.py), ensuring a diverse audio corpus for testing.
  - **Led the quantitative evaluation process**: Responsible for executing the automated testing pipeline via terminal and performing in-depth statistical analysis of key performance metrics, including Latency, SNR, and Compression Ratio.
  - Authored the project's technical documentation and README.md.
- **Hoang Thi Phuong Nga:**
  - Developed the interactive dashboard (app.py) specifically for **visualizing audio properties**, including the generation and display of real-time spectrograms and waveforms.
  - Implemented the **metrics computation module (metrics.py)** and the evaluation script structure (evaluate.py).
  - **Led the visual and comparative analysis**: Responsible for capturing and evaluating audio characteristics from the dashboard, including comparative analysis of spectrograms and waveforms across different codecs.
  - Developed the client-side streaming logic (client.py) and managed environment dependencies (requirements.txt)
- **Both authors:**
  - Collaborated on the overall system architecture design and pipeline definition.
  - Integrated all individual modules into a functional, end-to-end audio streaming system.
  - Designed the experimental framework (bitrate sweeps and cross-codec comparative studies).
  - Conducted comprehensive testing, debugging, and performance optimization.
  - Jointly performed data interpretation and finalized the technical report.

XII. TABLE OF FIGURES AND TABLES

FIGURES

Figure 1. Architecture Overview 7

Figure 2. SNR vs Bitrate - Sample.wav 15

Figure 3. Compression Ratio vs Birate - Sample.wav 16

Figure 4. Encode/Decode Latency @128kbps - Sample.wav 18

Figure 5. Spectrogram comparison of original and MP3 compressed signal at 128 kbps for Sample.wav 19

Figure 6. Spectrogram comparison of original and AAC compressed signal at 128 kbps for Sample.wav 19

Figure 7. Spectrogram comparison of original and OPUS compressed signal at 128 kbps for Sample.wav 20

Figure 8. Spectrogram comparison of original and OGG compressed signal at 128 kbps for Sample.wav 20

Figure 9. Waveform comparison of original and MP3-compressed speech signal at 128 kbps – Sample.wav 20

Figure 10. SNR vs Bitrate - Speech_test.wav 22

Figure 11. Compression Ratio vs Bitrate - Speech_test.wav 23

Figure 12. Encode/Decode Latency @128 kbps 25

Figure 13. Spectrogram comparison of original and MP3 compressed signal at 128 kbps for Speech_test.wav 26

Figure 14. Spectrogram comparison of original and AAC compressed signal at 128 kbps for Speech_test.wav 26

Figure 15. Spectrogram comparison of original and OPUS compressed signal at 128 kbps for Speech_test.wav 26

Figure 16. Spectrogram comparison of original and OGG compressed signal at 128 kbps for Speech_test.wav 27

Figure 17. Waveform comparison of original and MP3-compressed speech signal at 128 kbps - Speech_test.wav 27

Figure 18. SNR vs Bitrate - Music_test.wav 28

Figure 19. Compression Ratio vs Bitrate - Music_test.wav 30

Figure 20. Encode/Decode Latency @128 kbps - Music_test.wav 31

Figure 21. Spectrogram comparison of original and MP3 compressed signal at 128 kbps for Music_test.wav 32

Figure 22. Spectrogram comparison of original and AAC compressed signal at 128 kbps for Music_test.wav 33

Figure 23. Spectrogram comparison of original and OPUS compressed signal at 128 kbps for Music_test.wav 33

Figure 24. Spectrogram comparison of original and OGG compressed signal at 128 kbps for Music_test.wav 33

Figure 25. Waveform comparison of original and MP3-compressed speech signal at 128 kbps - Music_test.wav 34

Figure 26. SNR vs Bitrate - Percussion_test.wav 35

Figure 27. Compression Ratio vs Bitrate - Percussion_test.wav 37

Figure 28. Encode/Decode Latency @128 kbps - Percussion_test.wav 39

Figure 29. Spectrogram comparison of original and MP3 compressed signal at 128 kbps for Percussion_test.wav 40

Figure 30. Spectrogram comparison of original and AAC compressed signal at 128 kbps for Percussion_test.wav 40

Figure 31. Spectrogram comparison of original and OPUS compressed signal at 128 kbps for Percussion_test.wav 40

Figure 32. Spectrogram comparison of original and OGG compressed signal at 128 kbps for Percussion_test.wav 41

Figure 33. Waveform comparison of original and MP3-compressed speech signal at 128 kbps - Percussion_test.wav 41

Figure 34. SNR vs Bitrate - Noise_test.wav 43

Figure 35. Compression ratio vs Bitrate – Noise_test.wav 44

Figure 36. Encode/Decode Latency !128 kbps - Noise_test.wav 46

Figure 37. Spectrogram comparison of original and MP3 compressed signal at 128 kbps for Noise_test.wav 47

Figure 38. Spectrogram comparison of original and AAC compressed signal at 128 kbps for Noise_test.wav 48

Figure 39. Spectrogram comparison of original and OPUS compressed signal at 128 kbps for Noise_test.wav 48

Figure 40. Spectrogram comparison of original and OGG compressed signal at 128 kbps for Noise_test.wav 48

Figure 41. Waveform comparison of original and MP3-compressed speech signal at 128 kbps – Noise_test.wav 49

TABLES

Table 1. Evaluation Metrics Summary 12

Table 2. SNR (dB) Comparison across Codecs and Bitrates - Sample.wav 14

Table 3. Compression Ratio (x times reduction from original) - Sample.wav 16

Table 4. Processing Time (ms) for 10 Seconds of Audio - Sample.wav 17

Table 5. SNR (dB) Comparison across Codecs and Bitrates - Speech_test.wav 21

Table 6. Compression Ratio (x times reduction from original) - Speech_test.wav 23

Table 7. Processing Time (ms) for 10 Seconds of Audio - Speech_test.wav 24

Table 8. SNR (dB) Comparison across Codecs and Bitrates - Music_test.wav 28

Table 9. Compression Ratio (x times reduction from original) - Music_test.wav 29

Table 10. Processing Time (ms) for 10 Seconds of Audio - Music_test.wav 31

Table 11. SNR (dB) Comparison across Codecs and Bitrates Percussion_test.wav 34

Table 12. Compression Ratio (x times reduction from original) - Percussion_test.wav 36

Table 13. Processing Time (ms) for 10 Seconds of Audio Percussion_test.wav 38

Table 14. SNR (dB) Comparison across Codecs and Bitrates Noise_test.wav 42

Table 15. Compression Ratio (x times reduction from original) - Noise_test.wav 44

Table 16. Processing Time (ms) for 10 Seconds of Audio Noise_test.wav 45

XIII. SUPPLEMENTARY MATERIALS

The supplementary resources for this project are stored publicly and can be accessed via the following links:

Source Code Repository: ET4262E_168224_25219_GithubCode

Demonstration Video: ET4262E_168224_25219_DemoVideo

Presentation Slides: ET4262E_168224_25219_Slide

Dataset: ET4262E_168224_25219_Dataset